Dynamic Programming — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

353. Dynamic Programming Introduction

Understanding the Problem NEW

Dynamic Programming (DP) is a technique for speeding up recursive problems that recompute the **same subproblem** many times — instead of solving it fresh every time it's encountered, you solve it once and **remember (cache)** the answer for reuse. This chapter's whole theme is spotting 'overlapping subproblems' hiding inside a recursive solution and eliminating that redundant recomputation.

Brute Force Approach NEW

A plain recursive solution to a DP problem (like naive Fibonacci, computing fib(n) = fib(n-1) + fib(n-2)) recomputes the same smaller subproblems exponentially many times — e.g. fib(5) calls fib(3) twice, fib(2) three times, and so on, giving $O(2^n)$ time for something that has only n distinct subproblems.

Why We Optimize NEW

Two equivalent ways to eliminate the redundant recomputation: **Memoization** (top-down) keeps the natural recursive structure, but stores each subproblem's answer in an array/hashmap the first time it's computed, and simply returns the cached value on any later request instead of recomputing; **Tabulation** (bottom-up) instead builds up the answer iteratively from the smallest subproblems to the largest, filling a table in order, with no recursion at all. Both bring a problem like naive Fibonacci down from $O(2^n)$ to O(n) — this 'recognize repeated subproblems, cache instead of recompute' idea is the single unifying theme behind every problem in this chapter, however different they look on the surface.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding DP - optimization over recursion with overlapping subproblems.

Visual Diagram

```
Fibonacci: f(n) = f(n-1) + f(n-2)

       Recursion Tree (f(5)):
                              f(5)
```

```
                          /       \
                    f(4)            f(3)
                  /      \        /      \
              f(3)      f(2)    f(2)    f(1)
            /    \     /    \ /    \
          f(2) f(1) f(1) f(0) f(1) f(0)
        /    \
       f(1) f(0)

       DP Approaches:
       1. Memoization (Top-Down)
       2. Tabulation (Bottom-Up)
       3. Space Optimization

       DP Steps:
       1. Identify DP state
       2. Write recurrence
       3. Base cases
       4. Fill DP table / Memoize
```

Java Implementation

```
class Solution {

           // 1. Memoization (Top-Down)
           public int fibMemo(int n) {
               int[] dp = new int[n + 1];
               Arrays.fill(dp, -1);
               return fibHelper(n, dp);
           }

           private int fibHelper(int n, int[] dp) {
               if (n <= 1) return n;
               if (dp[n] != -1) return dp[n];
               dp[n] = fibHelper(n - 1, dp) + fibHelper(n - 2, dp);
               return dp[n];
           }

           // 2. Tabulation (Bottom-Up)
           public int fibTab(int n) {
               if (n <= 1) return n;
               int[] dp = new int[n + 1];
               dp[0] = 0;
               dp[1] = 1;
               for (int i = 2; i <= n; i++) {
                   dp[i] = dp[i - 1] + dp[i - 2];
               }
               return dp[n];
           }

           // 3. Space Optimized
           public int fibSpace(int n) {
```

```
            if (n <= 1) return n;
            int prev2 = 0, prev1 = 1;
            for (int i = 2; i <= n; i++) {
                int curr = prev1 + prev2;
                prev2 = prev1;
                prev1 = curr;
            }
            return prev1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) or O(1)

354. Climbing Stairs

Understanding the Problem NEW

Count the number of distinct ways to climb n stairs, if you can take either 1 or 2 steps at a time.

Brute Force Approach NEW

A direct recursive solution — `ways(n) = ways(n-1) + ways(n-2)` (since your last step was either a 1-step from stair n-1, or a 2-step from stair n-2) — looks correct but recomputes the same smaller values of `ways(k)` exponentially many times, exactly like naive Fibonacci, giving $O(2^n)$.

Why We Optimize NEW

This is literally the Fibonacci recurrence in disguise. Apply **memoization** (cache each `ways(k)` the first time it's computed) or **tabulation** (build up `ways(1), ways(2), ..., ways(n)` iteratively in an array) to eliminate the repeated recomputation, bringing it down to O(n) time (and this can be further reduced to O(1) space, since each answer only ever needs the previous two values, not the whole table) — the very first and simplest illustration of the memoization/tabulation idea introduced in problem 353.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Ways to climb n stairs taking 1 or 2 steps.

Visual Diagram

```
n = 4:
       Ways:
       1+1+1+1
       1+1+2
       1+2+1
       2+1+1
       2+2
       Total: 5

       DP: dp[i] = dp[i-1] + dp[i-2]
       dp[0] = 1
```

```
        dp[1] = 1
        dp[2] = 2
        dp[3] = 3
        dp[4] = 5
```

Solution Approach

Concept: Fibonacci pattern.

Algorithm

```
Algorithm CLIMB_STAIRS(n):
            If n <= 1: Return 1
            dp[0] = 1, dp[1] = 1
            For i = 2 to n:
                dp[i] = dp[i-1] + dp[i-2]
            Return dp[n]
```

Java Implementation

```
class Solution {
            public int climbStairs(int n) {
                if (n <= 1) return 1;
                int prev2 = 1, prev1 = 1;
                for (int i = 2; i <= n; i++) {
                    int curr = prev1 + prev2;
                    prev2 = prev1;
                    prev1 = curr;
                }
                return prev1;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

355. Frog Jump (DP-3)

Understanding the Problem NEW

A frog needs to jump across stones (positions 0 to n-1), able to jump either 1 or 2 stones ahead each time, paying a cost equal to the height difference between the stones it jumps between; find the minimum total cost to reach the last stone.

Brute Force Approach NEW

A direct recursion — `minCost(i) = min(minCost(i-1) + cost(i-1,i), minCost(i-2) + cost(i-2,i))` — again recomputes the same `minCost(k)` for smaller k exponentially many times if implemented naively without caching.

Why We Optimize NEW

Same fix as Climbing Stairs: memoize (or tabulate) `minCost(i)` for each stone, since each value only needs to be computed once and reused whenever a later stone's calculation needs it. This is essentially 'Climbing Stairs, but minimizing a cost instead of counting ways' — recognizing the shared recursive shape (each state depends only on the previous 1-2 states) is what tells you the same memoization/tabulation fix applies, turning $O(2^n)$ into O(n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum cost to reach last stone with jumps of 1 or 2.

Visual Diagram

```
Heights: [10, 30, 40, 20]
       Costs:
       dp[0] = 0
       dp[1] = |30-10| = 20
       dp[2] = min(|40-30| + dp[1], |40-10| + dp[0]) = min(10+20, 30+0) = 30
       dp[3] = min(|20-40| + dp[2], |20-30| + dp[1]) = min(20+30, 10+20) = 30
       Result: 30
```

Solution Approach

Concept: DP with two-step transitions.

Algorithm

```
Algorithm FROG_JUMP(heights):
           n = heights.length
           dp = new int[n]
           dp[0] = 0
           dp[1] = abs(heights[1] - heights[0])

           For i = 2 to n-1:
               one = dp[i-1] + abs(heights[i] - heights[i-1])
               two = dp[i-2] + abs(heights[i] - heights[i-2])
               dp[i] = min(one, two)

           Return dp[n-1]
```

Java Implementation

```
class Solution {
           public int frogJump(int[] heights) {
               int n = heights.length;
               int[] dp = new int[n];
               dp[0] = 0;
               dp[1] = Math.abs(heights[1] - heights[0]);

                 for (int i = 2; i < n; i++) {
                     int one = dp[i-1] + Math.abs(heights[i] - heights[i-1]);
                     int two = dp[i-2] + Math.abs(heights[i] - heights[i-2]);
                     dp[i] = Math.min(one, two);
                 }
                 return dp[n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

356. Frog Jump with K Distances (DP-4)

Understanding the Problem NEW

Same as Frog Jump, but now the frog can jump up to K stones ahead (not just 1 or 2), still minimizing total jump cost.

Brute Force Approach NEW

Direct recursion — `minCost(i) = min over j=1..K of (minCost(i-j) + cost(i-j, i))` — without caching, this recomputes shared subproblems exponentially, and even with caching, naively trying all K options at every position costs O(n×K) if not careful.

Why We Optimize NEW

Same memoization/tabulation fix as problem 355, generalized: cache `minCost(i)` for each position, computed by trying all K possible jump-back distances — achieving O(n×K) total (n positions, each considering up to K options), a direct generalization of the 2-choice version to a K-choice version, with the same underlying 'cache each position's answer once' principle.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Frog can jump up to k steps.

Visual Diagram

```
Heights: [10, 30, 40, 50, 20], k=3
       dp[i] = min over j=i-k..i-1 of dp[j] + abs(h[i] - h[j])
```

Solution Approach

Concept: DP with k-step transitions.

Algorithm

```
Algorithm FROG_JUMP_K(heights, k):
           n = heights.length
           dp = new int[n]
```

```
            dp[0] = 0

            For i = 1 to n-1:
                minCost = INF
                For j = max(0, i-k) to i-1:
                    minCost = min(minCost, dp[j] + abs(heights[i] - heights[j]))
                dp[i] = minCost

            Return dp[n-1]
```

Java Implementation

```
class Solution {
            public int frogJumpK(int[] heights, int k) {
                int n = heights.length;
                int[] dp = new int[n];
                dp[0] = 0;

                  for (int i = 1; i < n; i++) {
                      int minCost = Integer.MAX_VALUE;
                      for (int j = Math.max(0, i - k); j < i; j++) {
                          minCost = Math.min(minCost,
                              dp[j] + Math.abs(heights[i] - heights[j]));
                      }
                      dp[i] = minCost;
                  }
                  return dp[n-1];
            }
       }
```

Time & Space Complexity

Time Complexity: O(n*k) | Space Complexity: O(n)

357. Maximum Sum of Non-Adjacent Elements

Understanding the Problem NEW

Given an array, find the maximum sum achievable by selecting elements such that no two selected elements are adjacent to each other.

Brute Force Approach NEW

Try every possible subset that avoids adjacent pairs and compute sums — exponential (2^n possible 'valid or invalid' subsets to consider without a clear plan).

Why We Optimize NEW

At each position i, you face a simple binary choice: **either include** element i (in which case you must skip i-1, adding `arr[i] + bestSum(i-2)`) **or exclude** it (keeping `bestSum(i-1)` unchanged) — take whichever is larger. This recursive choice, memoized (or tabulated) per position, computes the answer in O(n) instead of exponential subset enumeration — and since each position only ever needs the previous two computed values, this can be further optimized to O(1) space, just like Climbing Stairs, by keeping two running variables instead of a full array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum sum without taking adjacent elements.

Visual Diagram

```
Array: [2, 1, 4, 9]
       dp[0] = 2
       dp[1] = max(2, 1) = 2
       dp[2] = max(dp[1], dp[0] + 4) = max(2, 2+4=6) = 6
       dp[3] = max(dp[2], dp[1] + 9) = max(6, 2+9=11) = 11
       Result: 11 (2+9)

       Recurrence: dp[i] = max(dp[i-1], dp[i-2] + arr[i])
```

Solution Approach

Concept: Include/exclude current element.

Algorithm

```
Algorithm MAX_NON_ADJACENT(arr):
           n = arr.length
           if n == 1: return arr[0]
           prev2 = arr[0]
           prev1 = max(arr[0], arr[1])

           For i = 2 to n-1:
               curr = max(prev1, prev2 + arr[i])
               prev2 = prev1
               prev1 = curr

           Return prev1
```

Java Implementation

```
class Solution {
           public int maxNonAdjacentSum(int[] nums) {
               int n = nums.length;
               if (n == 0) return 0;
               if (n == 1) return nums[0];

                 int prev2 = nums[0];
                 int prev1 = Math.max(nums[0], nums[1]);

                 for (int i = 2; i < n; i++) {
                     int curr = Math.max(prev1, prev2 + nums[i]);
                     prev2 = prev1;
                     prev1 = curr;
                 }
                 return prev1;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

358. House Robber (DP-6)

Understanding the Problem NEW

The exact same problem as Maximum Sum of Non-Adjacent Elements (357), just given a real-world story: a robber can't rob two adjacent houses, maximize total loot.

Brute Force Approach NEW

Same exponential subset enumeration as 357 without a clear DP plan.

Why We Optimize NEW

Identical technique to problem 357 — recognizing this is the same underlying DP recurrence wearing a different name is the entire point of including both versions back to back, reinforcing that many 'differently-named' problems are really the same recurrence relation.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum money without robbing adjacent houses.

Visual Diagram

```
Houses: [2, 7, 9, 3, 1]
      dp[0] = 2
      dp[1] = max(2,7) = 7
      dp[2] = max(7, 2+9=11) = 11
      dp[3] = max(11, 7+3=10) = 11
      dp[4] = max(11, 11+1=12) = 12
      Result: 12 (2+9+1)

      Same as max non-adjacent sum.
```

Solution Approach

Concept: Same as max non-adjacent sum.

Algorithm

```
Algorithm HOUSE_ROBBER(nums):
          return MAX_NON_ADJACENT(nums)
```

Java Implementation

```
class Solution {
          public int rob(int[] nums) {
              if (nums.length == 0) return 0;
              if (nums.length == 1) return nums[0];

                 int prev2 = nums[0];
                 int prev1 = Math.max(nums[0], nums[1]);

                 for (int i = 2; i < nums.length; i++) {
                     int curr = Math.max(prev1, prev2 + nums[i]);
                     prev2 = prev1;
                     prev1 = curr;
                 }
                 return prev1;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

359. Ninja's Training (DP-7)

Understanding the Problem NEW

A ninja trains over n days, choosing one of 3 possible activities each day (with associated points), but can't repeat the *same* activity on two consecutive days; maximize total points across all days.

Brute Force Approach NEW

Try every possible sequence of daily activity choices respecting the no-repeat-adjacent-day constraint — 3^n possible sequences, exponential.

Why We Optimize NEW

Track, for each day, the best achievable total points *given which activity was chosen the previous day* — a slightly richer DP state than the simple 'best sum so far' of House Robber, since the decision now depends on more than just 'did I pick the previous position or not.' At each day, for each of the 3 possible 'yesterday's activity' states, compute the best total by choosing any activity *except* yesterday's, using the previous day's already-computed best values for each of those combinations. This memoized/tabulated approach computes the answer in $O(n \times 3 \times 3)$ (a small constant multiplier per day) instead of the exponential 3^n sequence enumeration, by tracking a slightly larger 'state' (day, previous activity) rather than just 'day.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max points with no same activity on consecutive days.

Visual Diagram

```
Points: [[10,40,70], [20,50,80], [30,60,90]]
       dp[day][activity] = max points

       Day 0: [10,40,70]
       Day 1:
         activity 0: 20 + max(40,70) = 20+70=90
         activity 1: 50 + max(10,70) = 50+70=120
         activity 2: 80 + max(10,40) = 80+40=120
```

```
       Day 2:
         activity 0: 30 + max(120,120) = 150
         activity 1: 60 + max(90,120) = 180
         activity 2: 90 + max(90,120) = 210
       Result: 210
```

Solution Approach

Concept: 2D DP with activity constraint.

Algorithm

```
Algorithm NINJA_TRAINING(points):
           n = points.length
           dp = new int[n][3]
           dp[0] = points[0]

           For i = 1 to n-1:
               For j = 0 to 2:
                   For k = 0 to 2:
                       If k != j:
                           dp[i][j] = max(dp[i][j], dp[i-1][k] + points[i][j])

           Return max(dp[n-1])
```

Java Implementation

```
class Solution {
           public int ninjaTraining(int[][] points) {
               int n = points.length;
               int[][] dp = new int[n][3];
               dp[0] = points[0];

                 for (int i = 1; i < n; i++) {
                     for (int j = 0; j < 3; j++) {
                         for (int k = 0; k < 3; k++) {
                             if (k != j) {
                                 dp[i][j] = Math.max(dp[i][j], dp[i-1][k] +
points[i][j]);
                             }
                         }
                     }
                 }
                 return Math.max(dp[n-1][0], Math.max(dp[n-1][1], dp[n-1][2]));
           }

     }
```

Time & Space Complexity

Time Complexity: O(n*3*3) | Space Complexity: O(n*3)

360. Grid Unique Paths (DP-8)

Understanding the Problem NEW

Count the number of unique paths from the top-left to bottom-right of an m×n grid, moving only right or down at each step.

Brute Force Approach NEW

Try every possible sequence of right/down moves recursively and count how many reach the destination — exponential, since the recursive branching (right or down at every cell) explores many overlapping paths through shared intermediate cells repeatedly.

Why We Optimize NEW

The number of paths to reach any cell (r, c) is simply `paths(r-1, c) + paths(r, c-1)` (you must have arrived from either directly above or directly to the left) — memoizing (or tabulating, filling the grid row by row) this per-cell value means each cell's path-count is computed exactly once and reused by its neighbours below/right, achieving O(m×n) instead of the exponential path-by-path recursive enumeration. (There's also a purely combinatorial closed-form using binomial coefficients — choosing which of the (m+n-2) total moves are 'down' — but the DP approach generalizes more easily to variants like the next problem, which adds obstacles.)

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Number of ways to reach bottom-right from top-left.

Visual Diagram

```
Grid: 3x3
       Ways:
       [1,1,1]
       [1,2,3]
       [1,3,6]
       dp[i][j] = dp[i-1][j] + dp[i][j-1]
       Result: 6
```

```
       Combinatorics: C(m+n-2, m-1)
```

Solution Approach

Concept: DP with left and top cells.

Algorithm

```
Algorithm UNIQUE_PATHS(m, n):
           dp = new int[m][n]
           For i = 0 to m-1: dp[i][0] = 1
           For j = 0 to n-1: dp[0][j] = 1

            For i = 1 to m-1:
                For j = 1 to n-1:
                    dp[i][j] = dp[i-1][j] + dp[i][j-1]

            Return dp[m-1][n-1]
```

Java Implementation

```
class Solution {
           public int uniquePaths(int m, int n) {
               int[][] dp = new int[m][n];
               for (int i = 0; i < m; i++) dp[i][0] = 1;
               for (int j = 0; j < n; j++) dp[0][j] = 1;

                 for (int i = 1; i < m; i++) {
                     for (int j = 1; j < n; j++) {
                         dp[i][j] = dp[i-1][j] + dp[i][j-1];
                     }
                 }
                 return dp[m-1][n-1];
            }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

361. Grid Unique Paths 2 (DP-9)

Understanding the Problem NEW

Same as Unique Paths, but now some grid cells are obstacles that can't be passed through — count unique paths avoiding all obstacles.

Brute Force Approach NEW

Same exponential path enumeration as problem 360, now also needing to check at every step whether the path has hit an obstacle.

Why We Optimize NEW

Identical DP recurrence to problem 360, with one adjustment: any cell that's an obstacle simply has 0 paths through it (instead of summing from its neighbours, its value is forced to 0, and it doesn't contribute paths to cells after it). This shows how a DP recurrence can be trivially extended to handle a new constraint (obstacles) by adding one conditional check, without needing a fundamentally different algorithm — the underlying 'sum of ways from above and from the left' structure is unchanged.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Unique paths with obstacles.

Visual Diagram

```
Grid:
       [0,0,0]
       [0,1,0]
       [0,0,0]
       dp:
       [1,1,1]
       [1,0,1]
       [1,1,2]
       Result: 2

       If obstacle: dp[i][j] = 0
       Else: dp[i][j] = dp[i-1][j] + dp[i][j-1]
```

Solution Approach

Concept: DP with obstacle handling.

Algorithm

```
Algorithm UNIQUE_PATHS_OBSTACLE(grid):
           m = grid.length, n = grid[0].length
           if grid[0][0] == 1: return 0
           dp = new int[m][n]
           dp[0][0] = 1

           For i = 0 to m-1:
               For j = 0 to n-1:
                   If grid[i][j] == 1: dp[i][j] = 0
                   Else:
                       If i > 0: dp[i][j] += dp[i-1][j]
                       If j > 0: dp[i][j] += dp[i][j-1]

           Return dp[m-1][n-1]
```

Java Implementation

```
class Solution {
           public int uniquePathsWithObstacles(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               if (grid[0][0] == 1) return 0;

                 int[][] dp = new int[m][n];
                 dp[0][0] = 1;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (grid[i][j] == 1) {
                             dp[i][j] = 0;
                         } else {
                             if (i > 0) dp[i][j] += dp[i-1][j];
                             if (j > 0) dp[i][j] += dp[i][j-1];
                         }
                     }
                 }
                 return dp[m-1][n-1];

          }
      }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

362. Minimum Path Sum in Grid (DP-10)

Understanding the Problem NEW

Given a grid of costs, find the minimum sum path from top-left to bottom-right, moving only right or down.

Brute Force Approach NEW

Try every possible right/down path and sum costs, tracking the minimum — same exponential path enumeration as Unique Paths.

Why We Optimize NEW

Same recurrence shape as Unique Paths, with 'sum the ways' replaced by 'take the minimum': `minCost(r,c) = grid[r][c] + min(minCost(r-1,c), minCost(r,c-1))`, memoized/tabulated per cell. This directly reuses the grid-DP recurrence pattern from problems 360-361, just swapping the combination operation from addition (counting paths) to minimum (minimizing cost) — a recurring theme throughout this chapter: the same recursive 'shape' often applies to counting, summing, and optimizing versions of what's structurally the same problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum path sum from top-left to bottom-right.

Visual Diagram

```
Grid:
       [1,3,1]
       [1,5,1]
       [4,2,1]
       dp:
       [1,4,5]
       [2,7,6]
       [6,8,7]
       Result: 7 (1→3→1→1→1)

       dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])
```

Solution Approach

Concept: DP with min of left and top.

Algorithm

```
Algorithm MIN_PATH_SUM(grid):
           m = grid.length, n = grid[0].length
           dp = new int[m][n]
           dp[0][0] = grid[0][0]

           For i = 1 to m-1: dp[i][0] = dp[i-1][0] + grid[i][0]
           For j = 1 to n-1: dp[0][j] = dp[0][j-1] + grid[0][j]

           For i = 1 to m-1:
               For j = 1 to n-1:
                   dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1])

           Return dp[m-1][n-1]
```

Java Implementation

```
class Solution {
           public int minPathSum(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               int[][] dp = new int[m][n];
               dp[0][0] = grid[0][0];

                 for (int i = 1; i < m; i++) dp[i][0] = dp[i-1][0] + grid[i][0];
                 for (int j = 1; j < n; j++) dp[0][j] = dp[0][j-1] + grid[0][j];

                 for (int i = 1; i < m; i++) {
                     for (int j = 1; j < n; j++) {
                         dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1]);
                     }
                 }
                 return dp[m-1][n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

363. Minimum Path Sum in Triangular Grid (DP-11)

Understanding the Problem NEW

Given a triangular arrangement of numbers, find the minimum sum path from the top to the bottom row, where each step can move to one of the two numbers directly below the current position.

Brute Force Approach NEW

Try every possible path down the triangle and sum values — exponential (2 choices at each of n rows).

Why We Optimize NEW

Exactly the same 'minimum path sum' DP shape as problem 362, just on a triangular grid instead of a rectangular one: `minCost(r,c) = triangle[r][c] + min(minCost(r+1,c), minCost(r+1,c+1))` (computed bottom-up, since a triangle's natural 'end' is the single top cell, making bottom-up tabulation especially clean here), memoized/tabulated per position — same O(rows×cols) improvement over exponential path enumeration.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum path sum in triangular grid.

Visual Diagram

```
Triangle:
       [2]
       [3,4]
       [6,5,7]
       [4,1,8,3]

       dp:
       [2]
       [5,6]
       [11,10,13]
       [15,11,18,16]
       Result: 11
```

```
Bottom-up: dp[i][j] = triangle[i][j] + min(dp[i+1][j], dp[i+1][j+1])
```

Solution Approach

Concept: DP from bottom to top.

Algorithm

```
Algorithm MIN_PATH_TRIANGLE(triangle):
           n = triangle.size()
           dp = new int[n]
           dp = triangle[n-1]

           For i = n-2 down to 0:
               For j = 0 to i:
                   dp[j] = triangle[i][j] + Math.min(dp[j], dp[j+1])

           Return dp[0]
```

Java Implementation

```
class Solution {
           public int minimumTotal(List<List<Integer>> triangle) {
               int n = triangle.size();
               int[] dp = new int[n];
               for (int i = 0; i < n; i++) {
                   dp[i] = triangle.get(n-1).get(i);
               }

                 for (int i = n-2; i >= 0; i--) {
                     for (int j = 0; j <= i; j++) {
                         dp[j] = triangle.get(i).get(j) + Math.min(dp[j], dp[j+1]);
                     }
                 }
                 return dp[0];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

364. Minimum/Maximum Falling Path Sum (DP-12)

Understanding the Problem NEW

Given a grid, find the minimum (or maximum) sum path that starts anywhere in the top row and ends anywhere in the bottom row, moving down to any of the 3 cells directly below (down-left, straight down, down-right) at each step.

Brute Force Approach NEW

Try every possible path from any top-row starting cell through the grid — exponential, now with 3 choices per step instead of 2.

Why We Optimize NEW

Same grid-DP recurrence pattern as Minimum Path Sum (362), just with 3 possible 'sources' per cell instead of 2: `minCost(r,c) = grid[r][c] + min(minCost(r-1,c-1), minCost(r-1,c), minCost(r-1,c+1))`, and since the path can *start* anywhere in the top row, the final answer is the minimum value across the *entire* bottom row after filling the table — same O(rows×cols) improvement, extended to a slightly wider set of allowed moves per step.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum sum path falling from top to bottom.

Visual Diagram

```
Matrix:
       [2,1,3]
       [6,5,4]
       [7,8,9]

       dp[0] = [2,1,3]
       dp[1][0] = 6 + min(2,1) = 7
       dp[1][1] = 5 + min(2,1,3) = 6
       dp[1][2] = 4 + min(1,3) = 5
       dp[2][0] = 7 + min(7,6) = 13
       dp[2][1] = 8 + min(7,6,5) = 13
       dp[2][2] = 9 + min(6,5) = 14
       Result: 13
```

```
dp[i][j] = matrix[i][j] + min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])
```

Solution Approach

Concept: DP with three directions.

Algorithm

```
Algorithm MIN_FALLING_PATH(matrix):
           m = matrix.length, n = matrix[0].length
           dp = new int[m][n]
           dp[0] = matrix[0]

           For i = 1 to m-1:
               For j = 0 to n-1:
                   minVal = dp[i-1][j]
                   If j > 0: minVal = min(minVal, dp[i-1][j-1])
                   If j < n-1: minVal = min(minVal, dp[i-1][j+1])
                   dp[i][j] = matrix[i][j] + minVal

           Return min(dp[m-1])
```

Java Implementation

```
class Solution {
           public int minFallingPathSum(int[][] matrix) {
               int m = matrix.length, n = matrix[0].length;
               int[][] dp = new int[m][n];
               dp[0] = matrix[0];

                 for (int i = 1; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         int minVal = dp[i-1][j];
                         if (j > 0) minVal = Math.min(minVal, dp[i-1][j-1]);
                         if (j < n-1) minVal = Math.min(minVal, dp[i-1][j+1]);
                         dp[i][j] = matrix[i][j] + minVal;
                     }
                 }

                 int result = Integer.MAX_VALUE;
                 for (int val : dp[m-1]) {

                  result = Math.min(result, val);
              }
              return result;
          }
      }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

365. 3D DP: Ninja and His Friends (DP-13)

Understanding the Problem NEW

Two 'ninjas' start at the top corners of a grid and both move downward (each choosing among 3 directions per step, like problem 364) simultaneously, collecting values from cells they visit (a cell visited by both counts only once); maximize the total collected.

Brute Force Approach NEW

Try every possible combination of both ninjas' paths simultaneously — exponential in the combined branching of both ninjas' choices.

Why We Optimize NEW

Extend the DP **state** to track *both* ninjas' positions at once: since both move down one row per step together, the DP state is `(row, ninja1_column, ninja2_column)`, and at each state, try all 3×3=9 combinations of both ninjas' possible moves, taking the best. This is the chapter's first genuinely **3D DP** — the key generalization is recognizing that when a problem involves multiple 'agents' moving through the same structure, the DP state needs to track *all* of their positions together (not each independently), since their choices interact (the shared-cell rule). Still solvable in polynomial time — O(rows × cols² × 9) — despite the seemingly higher-dimensional complexity, by memoizing/tabulating over the combined state space rather than exploring the exponential combination of independent path choices.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Two ninjas collect maximum cherries.

Visual Diagram

```
Grid 3x3:
       [1,2,3]
       [4,5,6]
       [7,8,9]

       Two ninjas start from (0,0) and (0,1)
```

```
       Move together to bottom

       dp[step][i1][i2] = max cherries
       Same cell counted once
```

Solution Approach

Concept: 3D DP with same row movement.

Algorithm

```
Algorithm MAX_CHERRIES(grid):
           m = grid.length, n = grid[0].length
           dp = new int[m][n][n]

           For i = 0 to m-1:
               For j1 = 0 to n-1:
                   For j2 = 0 to n-1:
                       If j1 == j2:
                           dp[i][j1][j2] = grid[i][j1]
                       Else:
                           dp[i][j1][j2] = grid[i][j1] + grid[i][j2]

           For i = m-2 down to 0:
               For j1 = 0 to n-1:
                   For j2 = 0 to n-1:
                       maxVal = 0
                       For d1 = -1 to 1:
                           For d2 = -1 to 1:
                                If j1+d1 >=0 && j1+d1<n && j2+d2>=0 && j2+d2<n:
                                    maxVal = max(maxVal, dp[i+1][j1+d1][j2+d2])
                       dp[i][j1][j2] += maxVal
           Return dp[0][0][n-1]
```

Java Implementation

```
class Solution {
           public int maxCherries(int[][] grid) {
               int m = grid.length, n = grid[0].length;
               int[][][] dp = new int[m][n][n];

                 for (int j1 = 0; j1 < n; j1++) {
                     for (int j2 = 0; j2 < n; j2++) {
                         dp[m-1][j1][j2] = grid[m-1][j1] + (j1 == j2 ? 0 :
grid[m-1][j2]);
                     }
                 }

                 for (int i = m-2; i >= 0; i--) {
                     for (int j1 = 0; j1 < n; j1++) {
```

```
                        for (int j2 = 0; j2 < n; j2++) {
                            int maxVal = 0;
                            for (int d1 = -1; d1 <= 1; d1++) {
                                for (int d2 = -1; d2 <= 1; d2++) {
                                    int nj1 = j1 + d1;
                                    int nj2 = j2 + d2;
                                    if (nj1 >= 0 && nj1 < n && nj2 >= 0 && nj2 < n)
{
                                        maxVal = Math.max(maxVal, dp[i+1][nj1]
[nj2]);
                                    }
                                }
                            }
                            dp[i][j1][j2] = grid[i][j1] + (j1 == j2 ? 0 : grid[i]
[j2]) + maxVal;
                        }
                    }
                }
                return dp[0][0][n-1];
            }
        }
```

Time & Space Complexity

Time Complexity: O(m * n² * 9) | Space Complexity: O(m * n²)

366. Subset Sum Equal to Target (DP-14)

Understanding the Problem NEW

Given an array and a target sum, determine if *any* subset of the array sums to exactly that target.

Brute Force Approach NEW

Try every one of the 2^n possible subsets and check their sums — exponential, matching the include/exclude recursion tree from the Recursion chapter (problem 133), but now this same tree has massively overlapping subproblems worth caching.

Why We Optimize NEW

Define the DP state as `canMake(index, remainingTarget)` — can the elements from `index` onward form exactly `remainingTarget`? — with the same include/exclude recursive choice as the Recursion chapter's subset problems, but now **memoized** on the (index, remainingTarget) pair, since many different subset choices lead to the exact same (index, remainingTarget) state and would otherwise be recomputed repeatedly. This converts the exponential $O(2^n)$ recursion into O(n×target) — the first, and clearest, example in this chapter of a problem you already know how to solve recursively (from Chapter 6) becoming dramatically faster purely by adding memoization on top of the exact same recursive structure.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if any subset sums to target.

Visual Diagram

```
Array: [2, 3, 7, 8, 10], Target: 11

       dp[i][j] = possible with first i elements to get sum j
       dp:
             0 1 2 3 4 5 6 7 8 9 10 11
       0:   [T,F,F,F,F,F,F,F,F,F,F,F]
       1(2): [T,F,T,F,F,F,F,F,F,F,F,F]
```

```
       2(3): [T,F,T,T,F,T,F,F,F,F,F,F]
       ...
       Result: True

       Recurrence: dp[i][j] = dp[i-1][j] || dp[i-1][j-arr[i]]
```

Solution Approach

Concept: Include/exclude element.

Algorithm

```
Algorithm SUBSET_SUM(arr, target):
           n = arr.length
           dp = boolean[n+1][target+1]
           dp[0][0] = true

           For i = 1 to n:
               For j = 0 to target:
                   dp[i][j] = dp[i-1][j]
                   If j >= arr[i-1]:
                       dp[i][j] = dp[i][j] || dp[i-1][j-arr[i-1]]

           Return dp[n][target]
```

Java Implementation

```
class Solution {
           public boolean subsetSum(int[] arr, int target) {
               int n = arr.length;
               boolean[][] dp = new boolean[n+1][target+1];
               dp[0][0] = true;

                 for (int i = 1; i <= n; i++) {
                     for (int j = 0; j <= target; j++) {
                         dp[i][j] = dp[i-1][j];
                         if (j >= arr[i-1]) {
                             dp[i][j] = dp[i][j] || dp[i-1][j-arr[i-1]];
                         }
                     }
                 }
                 return dp[n][target];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(n*target)

367. Partition Equal Subset Sum (DP-15)

Understanding the Problem NEW

Given an array, determine if it can be partitioned into two subsets with *equal* sum.

Brute Force Approach NEW

Try every possible way to split the array into two groups and check if their sums match — exponential (2^n splits).

Why We Optimize NEW

This reduces directly to Subset Sum (366): the array can be split into two equal-sum halves if and only if **some subset sums to exactly half the total array sum** (if the total sum is odd, it's immediately impossible, since two equal integer halves couldn't sum to an odd total). Reusing the exact Subset Sum DP with target = totalSum/2 answers this directly, achieving O(n×totalSum) — another case of recognizing a new-sounding problem as a direct instance of one already solved.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Partition array into two subsets with equal sum.

Visual Diagram

```
Array: [1, 5, 11, 5]
       Sum = 22, target = 11
       Subset sum 11 exists: [1,5,5] = 11
       Result: True

       Array: [1, 2, 3, 5]
       Sum = 11, target = 5.5 (not integer)
       Result: False

       If total sum odd: return false
       Check subset sum = total/2
```

Solution Approach

Concept: Check subset sum = total/2.

Algorithm

```
Algorithm EQUAL_SUBSET(nums):
           total = sum(nums)
           If total % 2 != 0: Return false
           target = total / 2
           Return SUBSET_SUM(nums, target)
```

Java Implementation

```
class Solution {
           public boolean canPartition(int[] nums) {
               int total = 0;
               for (int num : nums) total += num;
               if (total % 2 != 0) return false;

                 int target = total / 2;
                 int n = nums.length;
                 boolean[] dp = new boolean[target + 1];
                 dp[0] = true;

                 for (int num : nums) {
                     for (int j = target; j >= num; j--) {
                         dp[j] = dp[j] || dp[j - num];
                     }
                 }
                 return dp[target];
           }
      }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(target)

368. Partition Set Into 2 Subsets With Minimum Difference (DP-16)

Understanding the Problem NEW

Given an array, partition it into two subsets such that the *difference* between their sums is as small as possible (not necessarily zero, unlike problem 367).

Brute Force Approach NEW

Try every possible split and compute the resulting difference, tracking the minimum — exponential.

Why We Optimize NEW

Extend Subset Sum (366) further: compute *which totals are achievable* by any subset (not just check one specific target, but find **every** achievable subset-sum up to the total array sum, using the same DP table but reading off every reachable value instead of just one). For every achievable subset-sum `s`, the resulting difference is `|totalSum - 2s|` (one subset sums to s, the other to totalSum-s); take the minimum of this over every achievable s. This reuses the exact same Subset Sum DP table, just extracting more information from it (every reachable sum, not just a yes/no for one target) — the same O(n×totalSum) table serves multiple related questions.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Partition array to minimize difference.

Visual Diagram

```
Array: [1, 2, 3, 4]
       Possible partitions:
       [1,4] and [2,3] → diff=0
       Result: 0

       Find sum closest to total/2.
```

Solution Approach

Concept: Subset sum closest to total/2.

Algorithm

```
Algorithm MIN_PARTITION_DIFF(arr):
           total = sum(arr)
           target = total / 2
           Find subset sum ≤ target closest to target
           diff = total - 2 * subsetSum
           Return diff
```

Java Implementation

```
class Solution {
           public int minPartitionDiff(int[] nums) {
               int total = 0;
               for (int num : nums) total += num;
               int target = total / 2;

                 boolean[] dp = new boolean[target + 1];
                 dp[0] = true;

                 for (int num : nums) {
                     for (int j = target; j >= num; j--) {
                         dp[j] = dp[j] || dp[j - num];
                     }
                 }

                 int closest = 0;
                 for (int i = target; i >= 0; i--) {
                     if (dp[i]) {
                         closest = i;
                         break;
                     }
                 }
                 return total - 2 * closest;
            }
      }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(target)

369. Count Subsets with Sum K (DP-17)

Understanding the Problem NEW

Given an array and a target, count *how many* subsets sum to exactly that target (not just whether one exists, unlike problem 366).

Brute Force Approach NEW

Enumerate every subset, check its sum, count matches — exponential.

Why We Optimize NEW

Same include/exclude DP state as Subset Sum (366), but instead of storing a true/false 'can this be achieved,' store a **count** — the number of ways to reach `remainingTarget` from `index` onward is the sum of the ways achieved by including vs. excluding the current element. Memoized/tabulated exactly like 366, just changing the combination step from OR (either way works) to addition (count both ways) — a small but important shift from 'existence' DP to 'counting' DP that reappears throughout this chapter (much like the Recursion chapter's print/count/check variations on the same tree, problem 134).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subsets with sum equal to target.

Visual Diagram

```
Array: [2, 3, 5, 6], Target: 8
       Subsets: [2,6], [3,5]
       Count: 2

       dp[i][j] = dp[i-1][j] + dp[i-1][j-arr[i]]
```

Solution Approach

Concept: Count ways with include/exclude.

Algorithm

```
Algorithm COUNT_SUBSETS(arr, target):
           n = arr.length
           dp = new int[n+1][target+1]
           dp[0][0] = 1

           For i = 1 to n:
               For j = 0 to target:
                   dp[i][j] = dp[i-1][j]
                   If j >= arr[i-1]:
                       dp[i][j] += dp[i-1][j-arr[i-1]]

           Return dp[n][target]
```

Java Implementation

```
class Solution {
           public int countSubsets(int[] nums, int target) {
               int n = nums.length;
               int[] dp = new int[target + 1];
               dp[0] = 1;

                 for (int num : nums) {
                     for (int j = target; j >= num; j--) {
                         dp[j] += dp[j - num];
                     }
                 }
                 return dp[target];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(target)

370. Count Partitions with Given Difference (DP-18)

Understanding the Problem NEW

Count the number of ways to partition the array into two subsets whose sums differ by exactly a given value.

Brute Force Approach NEW

Enumerate every split and check the resulting difference — exponential.

Why We Optimize NEW

Same algebraic reduction as problem 368, combined with the counting version from 369: if the two subsets differ by `diff` and sum to `totalSum` together, one subset's sum must be exactly `(totalSum + diff) / 2` (solving the two simultaneous equations) — so the answer is simply **Count Subsets with Sum K** (369) applied with this specific target. Another instance of reducing a new-sounding problem to an already-solved one via a bit of algebra, rather than designing a new DP from scratch.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count partitions with difference = diff.

Visual Diagram

```
Array: [1, 1, 2, 3], diff=1
       Partitions with diff=1:
       {1,3} and {1,2} → |4-3|=1
       {1,1,2} and {3} → |4-3|=1
       {1,1,3} and {2} → |5-2|=3
       Count: 2

       Formula: s1 - s2 = diff, s1 + s2 = total
       s1 = (total + diff) / 2
       Count subsets with sum = s1
```

Solution Approach

Concept: Reduce to subset sum count.

Algorithm

```
Algorithm COUNT_PARTITIONS(arr, diff):
           total = sum(arr)
           If (total + diff) % 2 != 0: Return 0
           target = (total + diff) / 2
           Return COUNT_SUBSETS(arr, target)
```

Java Implementation

```
class Solution {
           public int countPartitions(int[] nums, int diff) {
               int total = 0;
               for (int num : nums) total += num;

                 if ((total + diff) % 2 != 0) return 0;
                 int target = (total + diff) / 2;

                 int[] dp = new int[target + 1];
                 dp[0] = 1;

                 for (int num : nums) {
                     for (int j = target; j >= num; j--) {
                         dp[j] += dp[j - num];
                     }
                 }
                 return dp[target];
            }
       }
```

Time & Space Complexity

Time Complexity: O(n*target) | Space Complexity: O(target)

371. 0/1 Knapsack (DP-19)

Understanding the Problem NEW

The classic 0/1 Knapsack: given items with weights and values, and a knapsack with a maximum weight capacity, maximize total value, where **each item can be used at most once** (unlike Fractional Knapsack from the Greedy chapter, no partial items are allowed here, which is exactly why greedy doesn't work and DP is needed).

Brute Force Approach NEW

Try every one of the 2^n possible subsets of items and check which fit within the weight capacity while maximizing value — exponential, mirroring the same include/exclude recursion tree as Combination Sum (Recursion chapter, problem 137), but without the 'unlimited reuse' twist.

Why We Optimize NEW

DP state `bestValue(index, remainingCapacity)` : at each item, either **exclude** it (keep `bestValue(index+1, remainingCapacity)`) or, if it fits, **include** it (`item.value + bestValue(index+1, remainingCapacity - item.weight)`) — take the better option, memoized/tabulated over the (index, remainingCapacity) state space. This is the specific reason 0/1 Knapsack needs DP rather than the greedy 'sort by value/weight ratio' approach that worked for the Fractional version (Greedy chapter, problem 229) — since items can't be split, the greedy choice can be provably wrong here, and only exploring (and caching) the include/exclude decision space correctly finds the optimum, in O(n×capacity) instead of O(2^n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max value with weight capacity.

Visual Diagram

```
Items: weight=[1,3,4,5], value=[1,4,5,7], W=7
       dp[W] = max value
       dp[7] = max(dp[6] + 1, dp[4] + 4, dp[3] + 5, dp[2] + 7)
```

```
       Result: 11 (items 3+4: weight=4+3? Actually 5+4=9 >7)
       Correct: items 2+3: weight=3+4=7, value=4+5=9
```

Solution Approach

Concept: DP on capacity.

Algorithm

```
Algorithm KNAPSACK(weights, values, W):
           n = weights.length
           dp = new int[W+1]

           For i = 0 to n-1:
               For j = W down to weights[i]:
                   dp[j] = max(dp[j], dp[j-weights[i]] + values[i])

           Return dp[W]
```

Java Implementation

```
class Solution {
           public int knapsack(int[] weights, int[] values, int W) {
               int n = weights.length;
               int[] dp = new int[W + 1];

                 for (int i = 0; i < n; i++) {
                     for (int j = W; j >= weights[i]; j--) {
                         dp[j] = Math.max(dp[j], dp[j - weights[i]] + values[i]);
                     }
                 }
                 return dp[W];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*W) | Space Complexity: O(W)

372. Minimum Coins (DP-20)

Understanding the Problem NEW

Given coin denominations (unlimited supply of each) and a target amount, find the minimum number of coins needed to make that amount exactly — the same real-world problem as the Greedy chapter's Minimum Number of Coins (230), but now solved with DP so it's **correct for any denomination system**, not just 'well-behaved' ones.

Brute Force Approach NEW

Try every possible combination of coin counts summing to the target — exponential without structure, or the greedy approach from 230, which (as flagged there) can give a wrong answer for arbitrary denominations.

Why We Optimize NEW

DP state `minCoins(remainingAmount)`: try using each available denomination once, recursively solving for `remainingAmount - coin`, taking the best (fewest coins) option across all denominations tried — memoized/tabulated per remaining amount. Unlike the greedy approach, this DP is **always correct** regardless of the denomination system, since it genuinely explores every relevant combination (via caching, not blind greed) rather than assuming 'always take the largest coin that fits' is safe — directly resolving the caveat flagged back in the Greedy chapter's coin problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum coins to make amount.

Visual Diagram

```
Coins: [1,2,5], Amount: 11
       Min coins: 3 (5+5+1)

       dp[amount] = min coins
       dp[0] = 0
       dp[1] = 1
       dp[2] = 1
```

```
dp[3] = min(dp[2]+1, dp[1]+1) = 2
dp[4] = min(dp[2]+1, dp[1]+1) = 2
dp[5] = 1
dp[6] = min(dp[5]+1, dp[4]+1) = 2
dp[7] = min(dp[5]+1, dp[2]+1) = 2
dp[8] = min(dp[5]+1, dp[3]+1) = 3
dp[9] = min(dp[5]+1, dp[4]+1) = 3? Actually 5+2+2 = 3
dp[10] = 2
dp[11] = min(dp[10]+1, dp[9]+1, dp[6]+1) = 3
Result: 3
```

Solution Approach

Concept: DP with coin choices.

Algorithm

```
Algorithm MIN_COINS(coins, amount):
           dp = new int[amount+1]
           Arrays.fill(dp, INF)
           dp[0] = 0

           For i = 1 to amount:
               For coin in coins:
                   If coin <= i:
                       dp[i] = min(dp[i], dp[i-coin] + 1)

           Return dp[amount] == INF ? -1 : dp[amount]
```

Java Implementation

```
class Solution {
           public int minCoins(int[] coins, int amount) {
               int[] dp = new int[amount + 1];
               Arrays.fill(dp, Integer.MAX_VALUE);
               dp[0] = 0;

                 for (int i = 1; i <= amount; i++) {
                     for (int coin : coins) {
                         if (coin <= i && dp[i - coin] != Integer.MAX_VALUE) {
                             dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                         }
                     }
                 }
                 return dp[amount] == Integer.MAX_VALUE ? -1 : dp[amount];
           }
      }
```

Time & Space Complexity

Time Complexity: O(amount * coins) | Space Complexity: O(amount)

373. Target Sum (DP-21)

Understanding the Problem NEW

Given an array of numbers, assign either a '+' or '-' sign to each one so that the resulting sum equals a given target; count how many such sign-assignments achieve the target.

Brute Force Approach NEW

Try every one of the 2^n possible +/- sign combinations and check which sum to the target — exponential.

Why We Optimize NEW

A clever algebraic reduction turns this into Subset Sum-style counting: split the numbers into a 'positive group' P and 'negative group' N — then `sum(P) - sum(N) = target` and `sum(P) + sum(N) = totalSum`, which together mean `sum(P) = (totalSum + target) / 2`. So the answer is exactly **Count Subsets with Sum K** (369), with K set to this computed value — yet another instance of algebraically transforming a new-looking problem into the same Subset-Sum-counting DP already built, rather than inventing a new algorithm.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Assign +/- to get target sum.

Visual Diagram

```
Array: [1,1,1,1,1], Target: 3
       Ways: 5

       Formula: s1 - s2 = target, s1 + s2 = total
       s1 = (total + target) / 2
       Count subsets with sum = s1

       total = 5, target = 3
       s1 = (5+3)/2 = 4
       Count subsets sum 4 = 5
```

Solution Approach

Concept: Reduce to subset sum count.

Algorithm

```
Algorithm TARGET_SUM(nums, target):
           total = sum(nums)
           If (total + target) % 2 != 0: Return 0
           target = (total + target) / 2
           Return COUNT_SUBSETS(nums, target)
```

Java Implementation

```
class Solution {
           public int findTargetSumWays(int[] nums, int target) {
               int total = 0;
               for (int num : nums) total += num;

                 if ((total + target) % 2 != 0 || Math.abs(target) > total) return
0;
                 int sum = (total + target) / 2;

                 int[] dp = new int[sum + 1];
                 dp[0] = 1;

                 for (int num : nums) {
                     for (int j = sum; j >= num; j--) {
                         dp[j] += dp[j - num];
                     }
                 }
                 return dp[sum];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*sum) | Space Complexity: O(sum)

374. Coin Change 2 (DP-22)

Understanding the Problem NEW

Given coin denominations (unlimited supply) and a target amount, count the number of *distinct combinations* of coins that sum to the target (order doesn't matter — using one 5 then one 10 is the same combination as one 10 then one 5).

Brute Force Approach NEW

Try every possible combination of coin counts — exponential without structure, and naively trying every *ordering* of coin choices would overcount combinations that use the same coins in a different order.

Why We Optimize NEW

DP state `ways(coinIndex, remainingAmount)`: to avoid counting the same combination multiple times in different orders, process coin denominations **one at a time, in a fixed order**, deciding at each denomination 'how many of this coin to use' before moving to the next denomination — rather than trying combinations in every possible order. This 'process denominations in a fixed sequence, using each 0 or more times before moving on' structure is what correctly counts combinations instead of ordered sequences, achieving O(denominations × amount) via memoization/tabulation over the (coinIndex, remainingAmount) state.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Number of ways to make amount with coins (unlimited).

Visual Diagram

```
Coins: [1,2,5], Amount: 5
       Ways: 4
       5
       2+2+1
       2+1+1+1
       1+1+1+1+1
```

```
        dp[amount] = ways
        dp[0] = 1
        For coin in coins:
            For j = coin to amount:
                dp[j] += dp[j-coin]
```

Solution Approach

Concept: Unbounded knapsack count.

Algorithm

```
Algorithm COIN_CHANGE_2(coins, amount):
            dp = new int[amount+1]
            dp[0] = 1

            For coin in coins:
                For j = coin to amount:
                    dp[j] += dp[j-coin]

            Return dp[amount]
```

Java Implementation

```
class Solution {
            public int change(int amount, int[] coins) {
                int[] dp = new int[amount + 1];
                dp[0] = 1;

                  for (int coin : coins) {
                      for (int j = coin; j <= amount; j++) {
                          dp[j] += dp[j - coin];
                      }
                  }
                  return dp[amount];
            }
        }
```

Time & Space Complexity

Time Complexity: O(coins*amount) | Space Complexity: O(amount)

375. Unbounded Knapsack (DP-23)

Understanding the Problem NEW

Unbounded Knapsack: same as 0/1 Knapsack (371), but each item can now be used **unlimited** times instead of just once.

Brute Force Approach NEW

Same exponential exploration as 0/1 Knapsack without caching, now with an unbounded number of possible item-repetition combinations.

Why We Optimize NEW

Nearly identical DP recurrence to 0/1 Knapsack, with one crucial change: when you 'include' an item, you recurse back to the **same item index** (since it can be reused) rather than advancing to the next item — mirroring the exact same 'stay at the same candidate when reusing is allowed' pattern from Combination Sum in the Recursion chapter (problem 137). This single change to the recursive transition (stay vs. advance) is the entire difference between the bounded (0/1) and unbounded knapsack variants, both solved with the same O(n×capacity) memoized/tabulated DP shape.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max value with unlimited item usage.

Visual Diagram

```
Items: weight=[1,2,3], value=[10,15,40], W=6
       Max value: 80 (take 40 twice? weight 3*2=6, value 40*2=80)

       dp[W] = max value
       For each weight w:
           For j = w to W:
               dp[j] = max(dp[j], dp[j-w] + value)
```

Solution Approach

Concept: Unbounded knapsack.

Algorithm

```
Algorithm UNBOUNDED_KNAPSACK(weights, values, W):
           n = weights.length
           dp = new int[W+1]

           For i = 0 to n-1:
               For j = weights[i] to W:
                   dp[j] = max(dp[j], dp[j-weights[i]] + values[i])

           Return dp[W]
```

Java Implementation

```
class Solution {
           public int unboundedKnapsack(int[] weights, int[] values, int W) {
               int n = weights.length;
               int[] dp = new int[W + 1];

                 for (int i = 0; i < n; i++) {
                     for (int j = weights[i]; j <= W; j++) {
                         dp[j] = Math.max(dp[j], dp[j - weights[i]] + values[i]);
                     }
                 }
                 return dp[W];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n*W) | Space Complexity: O(W)

376. Rod Cutting Problem (DP-24)

Understanding the Problem NEW

Given a rod of length n and prices for each possible cut length, find the maximum total value obtainable by cutting the rod into pieces (each piece's value determined by its length) and selling them.

Brute Force Approach NEW

Try every possible way to partition the rod's length into pieces — exponential, since the number of ways to partition an integer grows very quickly.

Why We Optimize NEW

This is Unbounded Knapsack (375) in disguise: each possible cut-length is like an 'item' with a weight (the length) and a value (its price), and you're trying to fill a knapsack of capacity n using unlimited repetitions of any length — directly reusing the same unbounded-knapsack DP recurrence and complexity, O(n×n) here since capacity and number of 'items' (possible lengths) are both n. Recognizing that 'cut a rod for max value' and 'fill a knapsack unlimited times' are structurally the same problem is the entire insight, rather than deriving a new DP from scratch.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max value by cutting rod.

Visual Diagram

```
Length: 1,2,3,4
       Price: [2,5,7,8]
       Max value for length 4: 10 (2+2+2? Actually 4)
       Options:
       1+1+1+1 = 8
       2+2 = 10
       3+1 = 9
       4 = 8
       Result: 10
```

```
        dp[n] = max(dp[n], price[i] + dp[n-i-1])
```

Solution Approach

Concept: Unbounded knapsack with lengths.

Algorithm

```
Algorithm ROD_CUTTING(price, n):
            dp = new int[n+1]
            dp[0] = 0

            For i = 1 to n:
                For j = 1 to i:
                    dp[i] = max(dp[i], price[j-1] + dp[i-j])

            Return dp[n]
```

Java Implementation

```
class Solution {
            public int rodCutting(int[] price, int n) {
                int[] dp = new int[n + 1];
                dp[0] = 0;

                  for (int i = 1; i <= n; i++) {
                      for (int j = 1; j <= i; j++) {
                          dp[i] = Math.max(dp[i], price[j-1] + dp[i-j]);
                      }
                  }
                  return dp[n];
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

377. Longest Common Subsequence (DP-25)

Understanding the Problem NEW

Find the length of the Longest Common Subsequence (LCS) between two strings — the longest sequence of characters that appears in both strings in the same relative order (not necessarily contiguous). E.g. LCS of 'abcde' and 'ace' is 'ace,' length 3.

Brute Force Approach NEW

Try every one of the 2^n subsequences of the first string and check if each is also a subsequence of the second — exponential, and extremely wasteful given the massive overlap between different subsequence attempts.

Why We Optimize NEW

DP state `lcs(i, j)` — the LCS length considering only the first i characters of string 1 and first j characters of string 2: if the characters at positions i and j match, `lcs(i,j) = 1 + lcs(i-1,j-1)` (extend the match); otherwise, `lcs(i,j) = max(lcs(i-1,j), lcs(i,j-1))` (try dropping a character from either string). This **2D DP over two string indices** — a new kind of DP state shape compared to the single-index problems earlier in this chapter — is memoized/tabulated to give O(n×m) instead of exponential subsequence enumeration, and this exact 2D-index DP shape becomes the backbone for nearly every remaining string-comparison problem in this chapter (LCS variants, Edit Distance, Wildcard Matching).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Length of longest common subsequence.

Visual Diagram

```
s1 = "abcde", s2 = "ace"
       LCS = "ace" length=3

       dp[i][j] = LCS length for first i of s1, first j of s2
       If s1[i-1] == s2[j-1]: dp[i][j] = 1 + dp[i-1][j-1]
       Else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

```
        "" a c e
    ""  0 0 0 0
    a   0 1 1 1
    b   0 1 1 1
    c   0 1 2 2
    d   0 1 2 2
    e   0 1 2 3
```

Solution Approach

Concept: DP with matching and skipping.

Algorithm

```
Algorithm LCS(s1, s2):
          m = s1.length, n = s2.length
          dp = int[m+1][n+1]

           For i = 1 to m:
               For j = 1 to n:
                   If s1[i-1] == s2[j-1]:
                       dp[i][j] = 1 + dp[i-1][j-1]
                   Else:
                       dp[i][j] = max(dp[i-1][j], dp[i][j-1])

           Return dp[m][n]
```

Java Implementation

```
class Solution {
          public int lcs(String s1, String s2) {
              int m = s1.length(), n = s2.length();
              int[][] dp = new int[m+1][n+1];

                for (int i = 1; i <= m; i++) {
                    for (int j = 1; j <= n; j++) {
                        if (s1.charAt(i-1) == s2.charAt(j-1)) {
                            dp[i][j] = 1 + dp[i-1][j-1];
                        } else {
                            dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                        }
                    }
                }
                return dp[m][n];
           }

     }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

378. Print Longest Common Subsequence (DP-26)

Understanding the Problem NEW

Same as LCS, but now print the actual longest common subsequence string, not just its length.

Brute Force Approach NEW

You could try every subsequence and check for the longest match directly — exponential, same as plain LCS without a plan for reconstruction.

Why We Optimize NEW

First compute the full LCS DP table as in problem 377, then **walk backward through the table** from `lcs(n,m)`: whenever the characters at the current positions match, that character is part of the LCS (add it to the front of the result and move diagonally back); otherwise, move in whichever direction (i-1 or j-1) the table shows the value came from. This backward-reconstruction-from-a-filled-DP-table technique — compute the table forward, then trace an optimal path backward through it to recover the actual solution, not just its value — is a general technique that applies to many DP problems where you need the actual answer, not just its length/value/count.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print the LCS string.

Visual Diagram

```
s1 = "abcde", s2 = "ace"
       LCS = "ace"

       Backtrack:
       dp[5][3] = 3 → s1[4]=e, s2[2]=e match → include 'e'
       dp[4][2] = 2 → s1[3]=d, s2[1]=c no match → go left (dp[4][1]=2)
       dp[4][1] = 2 → s1[3]=d, s2[0]=a no match → go left (dp[3][1]=1)
       dp[3][1] = 1 → s1[2]=c, s2[0]=a no match → go up (dp[2][1]=1)
       dp[2][1] = 1 → s1[1]=b, s2[0]=a no match → go up (dp[1][1]=1)
```

```
       dp[1][1] = 1 → s1[0]=a, s2[0]=a match → include 'a'
       Result: "ace"
```

Solution Approach

Concept: Backtrack from DP table.

Algorithm

```
Algorithm PRINT_LCS(s1, s2, dp):
           i = m, j = n
           lcs = []

            While i > 0 && j > 0:
                If s1[i-1] == s2[j-1]:
                    lcs.add(s1[i-1])
                    i--; j--
                Else If dp[i-1][j] > dp[i][j-1]:
                    i--
                Else:
                    j--

            Return reverse(lcs)
```

Java Implementation

```
class Solution {
           public String printLCS(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];

                 // Fill DP table
                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = 1 + dp[i-1][j-1];
                         } else {
                             dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                         }
                     }
                 }

                 // Backtrack
                 StringBuilder result = new StringBuilder();
                 int i = m, j = n;
                 while (i > 0 && j > 0) {

                  if (s1.charAt(i-1) == s2.charAt(j-1)) {
                      result.append(s1.charAt(i-1));
                      i--; j--;
```

```
                } else if (dp[i-1][j] > dp[i][j-1]) {
                    i--;
                } else {
                    j--;
                }
            }
            return result.reverse().toString();
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

379. Longest Common Substring (DP-27)

Understanding the Problem NEW

Find the length of the Longest Common **Substring** (not subsequence) — a contiguous run of characters common to both strings, unlike LCS which allows gaps.

Brute Force Approach NEW

Check every possible substring of string 1 against every possible substring of string 2 for a match — O(n²×m²) or worse.

Why We Optimize NEW

Nearly the same 2D DP table shape as LCS (377), but with one crucial change: since a substring must be *contiguous*, a mismatch can't just 'skip' a character the way LCS does — instead, `dp(i,j) = 1 + dp(i-1,j-1)` on a match, but `dp(i,j) = 0` entirely on a mismatch (breaking the current run rather than trying to continue it via skips), and the final answer is the **maximum value found anywhere in the table** (not just the bottom-right corner, since the longest common substring could end at any position). This small but important change — reset to 0 on mismatch instead of taking a max of skip-options — is what converts the LCS recurrence into the Longest Common Substring recurrence.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Length of longest common substring.

Visual Diagram

```
s1 = "abcd", s2 = "bcde"
       LCS substring = "bcd" length=3

       If match: dp[i][j] = 1 + dp[i-1][j-1]
       Else: dp[i][j] = 0
       Track max

            "" b c d e
       ""   0 0 0 0 0
       a    0 0 0 0 0
```

```
       b    0 1 0 0 0
       c    0 0 2 0 0
       d    0 0 0 3 0
```

Solution Approach

Concept: DP with match only.

Algorithm

```
Algorithm LCS_SUBSTRING(s1, s2):
           m = s1.length, n = s2.length
           dp = int[m+1][n+1]
           maxLen = 0

            For i = 1 to m:
                For j = 1 to n:
                    If s1[i-1] == s2[j-1]:
                        dp[i][j] = 1 + dp[i-1][j-1]
                        maxLen = max(maxLen, dp[i][j])

            Return maxLen
```

Java Implementation

```
class Solution {
           public int longestCommonSubstring(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];
               int maxLen = 0;

                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = 1 + dp[i-1][j-1];
                             maxLen = Math.max(maxLen, dp[i][j]);
                         }
                     }
                 }
                 return maxLen;
            }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

380. Longest Palindromic Subsequence (DP-28)

Understanding the Problem NEW

Find the length of the Longest Palindromic Subsequence within a single string — the longest subsequence that reads the same forwards and backwards.

Brute Force Approach NEW

Try every one of the 2^n subsequences and check which are palindromes, tracking the longest — exponential.

Why We Optimize NEW

A clever reduction: the Longest Palindromic Subsequence of a string is exactly the **Longest Common Subsequence (377) between the string and its own reverse** — since any subsequence common to both the string and its reverse must, by construction, read the same both ways. This directly reuses the LCS DP with no modification needed beyond feeding it the string and its reverse as the two inputs — another strong example of reducing a new problem to one already solved, this time via a clever observation rather than a direct structural similarity.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Length of longest palindromic subsequence.

Visual Diagram

```
s = "bbbab"
       LPS = "bbbb" length=4

       dp[i][j] = LPS length for s[i..j]
       If i == j: dp[i][j] = 1
       If s[i] == s[j]: dp[i][j] = 2 + dp[i+1][j-1]
       Else: dp[i][j] = max(dp[i+1][j], dp[i][j-1])

       For "bbbab":
       dp[0][4] = 4 (bbbb)
```

Solution Approach

Concept: DP on intervals.

Algorithm

```
Algorithm LPS(s):
           n = s.length()
           dp = int[n][n]

           For i = 0 to n-1: dp[i][i] = 1

           For len = 2 to n:
               For i = 0 to n-len:
                   j = i + len - 1
                   If s[i] == s[j]:
                       dp[i][j] = 2 + (len == 2 ? 0 : dp[i+1][j-1])
                   Else:
                       dp[i][j] = max(dp[i+1][j], dp[i][j-1])

           Return dp[0][n-1]
```

Java Implementation

```
class Solution {
           public int longestPalindromeSubseq(String s) {
               int n = s.length();
               int[][] dp = new int[n][n];

                 for (int i = 0; i < n; i++) dp[i][i] = 1;

                 for (int len = 2; len <= n; len++) {
                     for (int i = 0; i <= n - len; i++) {
                         int j = i + len - 1;
                         if (s.charAt(i) == s.charAt(j)) {
                             dp[i][j] = 2 + (len == 2 ? 0 : dp[i+1][j-1]);
                         } else {
                             dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                         }
                     }
                 }
                 return dp[0][n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

381. Minimum Insertions to Make Palindrome (DP-29)

Understanding the Problem NEW

Find the minimum number of character insertions needed to turn a given string into a palindrome.

Brute Force Approach NEW

Try every possible way of inserting characters and check which produces a palindrome with the fewest insertions — exponential without structure.

Why We Optimize NEW

Another elegant reduction: the minimum insertions needed equals `string length - Longest Palindromic Subsequence length` (380) — because the LPS represents the 'already palindromic core' you can keep as-is, and every character *not* part of that core needs a matching insertion elsewhere to become symmetric. Directly reusing the LPS DP (which itself reuses LCS) to answer this, rather than solving insertion-counting from scratch, is the whole insight — reinforcing how a handful of core DP recurrences (LCS being the most reused one) power a surprising number of seemingly distinct string problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum insertions to make string palindrome.

Visual Diagram

```
s = "mbadm"
       LPS = "madam" (or "mbdm" no, "mam" length 3)
       Minimum insertions = n - LPS = 5 - 3 = 2

       "mbadm" → "madam" insert 'a' at pos 1? "mbda"...
       Actually: "mbadm" → "mabdamb"? Too complex.

       Formula: minInsertions = n - LPS(s)
```

Solution Approach

Concept: LPS length and total length.

Algorithm

```
Algorithm MIN_INSERTIONS(s):
           n = s.length()
           lps = LPS(s)
           Return n - lps
```

Java Implementation

```
class Solution {
           public int minInsertions(String s) {
               int n = s.length();
               int lps = longestPalindromeSubseq(s);
               return n - lps;
           }

           private int longestPalindromeSubseq(String s) {
               int n = s.length();
               int[][] dp = new int[n][n];

                 for (int i = 0; i < n; i++) dp[i][i] = 1;

                 for (int len = 2; len <= n; len++) {
                     for (int i = 0; i <= n - len; i++) {
                         int j = i + len - 1;
                         if (s.charAt(i) == s.charAt(j)) {
                             dp[i][j] = 2 + (len == 2 ? 0 : dp[i+1][j-1]);
                         } else {
                             dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                         }
                     }
                 }
                 return dp[0][n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

382. Minimum Insertions/Deletions to Convert (DP-30)

Understanding the Problem NEW

Find the minimum number of insertions and deletions needed to convert one string into another.

Brute Force Approach NEW

Try every possible sequence of insert/delete operations — exponential without structure.

Why We Optimize NEW

Reduces directly to LCS (377): whatever characters are part of the LCS between the two strings can stay untouched; every other character in string 1 must be **deleted** (`len(s1) - LCS` deletions), and every other character in string 2 must be **inserted** (`len(s2) - LCS` insertions). Computing LCS once and doing simple arithmetic on the lengths answers this directly — yet another problem that reduces cleanly to the same LCS computation, rather than needing a new DP recurrence.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum operations to convert s1 to s2.

Visual Diagram

```
s1 = "heap", s2 = "pea"
       Delete 'h','p' and insert 'p'? Actually:
       "heap" → "pea"
       Delete 'h', delete 'p', insert 'p'?
       Minimum: 3

       Formula:
       LCS = "ea" length=2
       Deletions = m - LCS = 4-2=2
       Insertions = n - LCS = 3-2=1
       Total = 3
```

Solution Approach

Concept: LCS approach.

Algorithm

```
Algorithm MIN_OPERATIONS(s1, s2):
           lcs = LCS(s1, s2)
           deletions = s1.length - lcs
           insertions = s2.length - lcs
           Return deletions + insertions
```

Java Implementation

```
class Solution {
           public int minOperations(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int lcs = longestCommonSubsequence(s1, s2);
               return (m - lcs) + (n - lcs);
           }

           private int longestCommonSubsequence(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];

                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = 1 + dp[i-1][j-1];
                         } else {
                             dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                         }
                     }
                 }
                 return dp[m][n];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

383. Shortest Common Supersequence (DP-31)

Understanding the Problem NEW

Find the Shortest Common Supersequence (SCS) — the shortest string that contains both given strings as subsequences.

Brute Force Approach NEW

Try building candidate supersequences directly and check if both strings are subsequences of each — without structure, this is a huge, poorly-defined search space.

Why We Optimize NEW

The SCS is built by **merging the two strings along their LCS (377)**: characters that are part of the LCS only need to appear once in the result, while characters *not* part of the LCS from either string must all appear (since they need to be 'passed through' individually). Concretely, reconstruct using the same backward-walk technique as Print LCS (378), but this time, when characters don't match, include *both* mismatched characters in the result (rather than skipping) before continuing the backward walk. This is a direct extension of the LCS-reconstruction technique from problem 378, adapted to build a merged supersequence instead of just extracting the common part.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find shortest string containing both as subsequences.

Visual Diagram

```
s1 = "abac", s2 = "cab"
       SCS = "cabac" or "caba"?
       LCS = "ab" length=2
       SCS length = m + n - LCS = 4+3-2=5

       Build SCS:
       1. Take LCS "ab"
       2. Add remaining chars in order
       Result: "cabac" (c + ab + ac)
```

Solution Approach

Concept: LCS based construction.

Algorithm

```
Algorithm SHORTEST_COMMON_SUPERSEQUENCE(s1, s2):
           lcs = LCS(s1, s2)
           Build SCS by merging s1 and s2 with LCS
```

Java Implementation

```
class Solution {
           public String shortestCommonSupersequence(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];

                 // Fill LCS DP
                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = 1 + dp[i-1][j-1];
                         } else {
                             dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                         }
                     }
                 }

                 // Build SCS
                 StringBuilder result = new StringBuilder();
                 int i = m, j = n;
                 while (i > 0 && j > 0) {
                     if (s1.charAt(i-1) == s2.charAt(j-1)) {
                         result.append(s1.charAt(i-1));
                         i--; j--;
                     } else if (dp[i-1][j] > dp[i][j-1]) {
                         result.append(s1.charAt(i-1));
                         i--;
                     } else {
                         result.append(s2.charAt(j-1));
                         j--;
                     }
                 }

                   while (i > 0) result.append(s1.charAt(--i));

              while (j > 0) result.append(s2.charAt(--j));

              return result.reverse().toString();
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

384. Distinct Subsequences (DP-32)

Understanding the Problem NEW

Count the number of distinct ways string T appears as a subsequence within string S. E.g. S='rabbbit', T='rabbit' → 3 ways (choosing different 'b's).

Brute Force Approach NEW

Try every one of the 2^|S| subsequences of S and check how many equal T — exponential.

Why We Optimize NEW

DP state `ways(i, j)` — number of ways to form the first j characters of T using the first i characters of S: if `S[i] == T[j]`, you have **two choices** — use this S character to match T (`ways(i-1, j-1)`) or skip it and still try to match using earlier S characters (`ways(i-1, j)`) — sum both; if they don't match, you must skip the S character (`ways(i-1, j)` only). This 2D DP (same index-pair shape as LCS, but counting instead of taking max/length) achieves O(n×m) instead of exponential subsequence enumeration, another instance of the 'counting variant' of a familiar 2D string-DP recurrence.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count distinct subsequences of s matching t.

Visual Diagram

```
s = "rabbbit", t = "rabbit"
       Distinct subsequences: 3

       dp[i][j] = ways using first i of s to form first j of t
       If match: dp[i][j] = dp[i-1][j] + dp[i-1][j-1]
       Else: dp[i][j] = dp[i-1][j]

           "" r a b b i t
       "" 1 0 0 0 0 0 0
       r  1 1 0 0 0 0 0
       a  1 1 1 0 0 0 0
       b  1 1 1 1 0 0 0
```

```
       b   1 1 1 2 0 0 0
       i   1 1 1 2 2 0 0
       t   1 1 1 2 2 2 0?
       Result: 3
```

Solution Approach

Concept: DP with match inclusion.

Algorithm

```
Algorithm DISTINCT_SUBSEQUENCES(s, t):
           m = s.length(), n = t.length()
           dp = int[m+1][n+1]
           dp[i][0] = 1

           For i = 1 to m:
               For j = 1 to n:
                   dp[i][j] = dp[i-1][j]
                   If s[i-1] == t[j-1]:
                       dp[i][j] += dp[i-1][j-1]

           Return dp[m][n]
```

Java Implementation

```
class Solution {
           public int numDistinct(String s, String t) {
               int m = s.length(), n = t.length();
               int[] dp = new int[n + 1];
               dp[0] = 1;

                 for (int i = 1; i <= m; i++) {
                     for (int j = n; j >= 1; j--) {
                         if (s.charAt(i-1) == t.charAt(j-1)) {
                             dp[j] += dp[j-1];
                         }
                     }
                 }
                 return dp[n];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(n)

385. Edit Distance (DP-33)

Understanding the Problem NEW

Find the minimum number of operations (insert, delete, or replace a character) needed to convert one string into another — a generalization of problem 382 that also allows character replacement, not just insert/delete.

Brute Force Approach NEW

Try every possible sequence of edit operations — exponential without structure.

Why We Optimize NEW

DP state `editDist(i, j)`: if the current characters match, no operation is needed here (`editDist(i-1, j-1)`); if they don't match, try all three operations and take the cheapest — **insert** (`1 + editDist(i, j-1)`), **delete** (`1 + editDist(i-1, j)`), or **replace** (`1 + editDist(i-1, j-1)`) — memoized/tabulated over the same 2D (i,j) index-pair shape used throughout this string-DP family. This is the most general string-transformation DP in the chapter (subsuming problem 382's insert/delete-only version as a special case where replacement is simply never the cheapest option), achieving O(n×m) instead of exponential operation-sequence exploration.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum operations to convert s1 to s2.

Visual Diagram

```
s1 = "horse", s2 = "ros"
       Operations: 3
       "horse" → "rorse" (replace h→r)
       "rorse" → "rose" (delete r)
       "rose" → "ros" (delete e)

       dp[i][j] = min operations
       If match: dp[i][j] = dp[i-1][j-1]
       Else: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
```

Solution Approach

Concept: DP with delete/insert/replace.

Algorithm

```
Algorithm EDIT_DISTANCE(s1, s2):
           m = s1.length, n = s2.length
           dp = int[m+1][n+1]

           For i = 0 to m: dp[i][0] = i
           For j = 0 to n: dp[0][j] = j

           For i = 1 to m:
               For j = 1 to n:
                   If s1[i-1] == s2[j-1]:
                       dp[i][j] = dp[i-1][j-1]
                   Else:
                       dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])

           Return dp[m][n]
```

Java Implementation

```
class Solution {
           public int minDistance(String s1, String s2) {
               int m = s1.length(), n = s2.length();
               int[][] dp = new int[m+1][n+1];

                 for (int i = 0; i <= m; i++) dp[i][0] = i;
                 for (int j = 0; j <= n; j++) dp[0][j] = j;

                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         if (s1.charAt(i-1) == s2.charAt(j-1)) {
                             dp[i][j] = dp[i-1][j-1];
                         } else {
                             dp[i][j] = 1 + Math.min(dp[i-1][j],
                                           Math.min(dp[i][j-1], dp[i-1][j-1]));
                         }
                     }
                 }
                 return dp[m][n];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

386. Wildcard Matching (DP-34)

Understanding the Problem NEW

Determine if a pattern (containing literal characters, '?' meaning any single character, and '*' meaning any sequence of characters, including empty) matches a given string entirely.

Brute Force Approach NEW

Try every possible way '*' characters could expand and check if the resulting expanded pattern matches the string exactly — exponential, since each '*' could expand to many different lengths.

Why We Optimize NEW

Same 2D DP index-pair shape as Edit Distance/LCS, but with match-checking logic tailored to wildcards: `match(i,j)` is true if the current pattern character is '?' or matches the string character exactly (then check `match(i-1,j-1)`); if it's '*', it can either match zero characters (check `match(i,j-1)`, moving past the '*' in the pattern) or match one more string character while staying on the same '*' (check `match(i-1,j)`) — take true if *either* option works. This reuses the same 2D-index DP structure as the rest of the string family, with the recurrence's match-logic adapted specifically to handle the two wildcard types, rather than exhaustively trying every possible '*' expansion length.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Wildcard matching with '*' and '?'.

Visual Diagram

```
s = "adceb", p = "*a*b"
       Result: true

       dp[i][j] = pattern matches s[0..i-1]
       If p[j-1] == '?': dp[i][j] = dp[i-1][j-1]
       If p[j-1] == '*': dp[i][j] = dp[i][j-1] || dp[i-1][j]
       If chars match: dp[i][j] = dp[i-1][j-1]
```

Solution Approach

Concept: DP with wildcard handling.

Algorithm

```
Algorithm WILDCARD_MATCH(s, p):
           m = s.length, n = p.length
           dp = boolean[m+1][n+1]
           dp[0][0] = true

            For j = 1 to n:
                If p[j-1] == '*': dp[0][j] = dp[0][j-1]

            For i = 1 to m:
                For j = 1 to n:
                    If p[j-1] == '?':
                        dp[i][j] = dp[i-1][j-1]
                    Else If p[j-1] == '*':
                        dp[i][j] = dp[i][j-1] || dp[i-1][j]
                    Else:
                        dp[i][j] = s[i-1] == p[j-1] && dp[i-1][j-1]

            Return dp[m][n]
```

Java Implementation

```
class Solution {
           public boolean isMatch(String s, String p) {
               int m = s.length(), n = p.length();
               boolean[][] dp = new boolean[m+1][n+1];
               dp[0][0] = true;

                 for (int j = 1; j <= n; j++) {
                     if (p.charAt(j-1) == '*') {
                         dp[0][j] = dp[0][j-1];
                     }
                 }

                 for (int i = 1; i <= m; i++) {
                     for (int j = 1; j <= n; j++) {
                         char sc = s.charAt(i-1);
                         char pc = p.charAt(j-1);

                         if (pc == '?') {
                             dp[i][j] = dp[i-1][j-1];
                         } else if (pc == '*') {

                          dp[i][j] = dp[i][j-1] || dp[i-1][j];
                      } else {
                          dp[i][j] = sc == pc && dp[i-1][j-1];
```

```
                    }
                }
            }
            return dp[m][n];
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

387. Best Time to Buy and Sell Stock I (DP-35)

Understanding the Problem NEW

Given daily stock prices, find the maximum profit from **one** buy and one sell transaction — the exact same problem as the Arrays chapter's Stock Buy and Sell (problem 27), revisited here to connect it to the DP family of stock problems that follow.

Brute Force Approach NEW

Check every pair of buy-day/sell-day combinations — $O(n^2)$, same as the naive approach flagged back in the Arrays chapter.

Why We Optimize NEW

As established back in problem 27: track a running minimum price seen so far, and at each day compute `currentPrice - minSoFar`, keeping the best — O(n) time, O(1) space, no DP table actually needed for this simplest single-transaction version. It's included here specifically as the 'base case' anchor for the more complex multi-transaction stock variants (388-392) that follow, each adding one more layer of complexity on top of this same core idea.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with one transaction.

Visual Diagram

```
Prices: [7,1,5,3,6,4]
       Max profit: 5 (buy at 1, sell at 6)

       Track min price and max profit.
```

Solution Approach

Concept: Single pass tracking min.

Algorithm

```
Algorithm MAX_PROFIT(prices):
           minPrice = INF
           maxProfit = 0

           For price in prices:
               minPrice = min(minPrice, price)
               maxProfit = max(maxProfit, price - minPrice)

           Return maxProfit
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int minPrice = Integer.MAX_VALUE;
               int maxProfit = 0;

                 for (int price : prices) {
                     minPrice = Math.min(minPrice, price);
                     maxProfit = Math.max(maxProfit, price - minPrice);
                 }
                 return maxProfit;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

388. Buy and Sell Stock II (DP-36)

Understanding the Problem NEW

Same stock setup, but now **unlimited transactions** are allowed (buy and sell multiple times, though you must sell before buying again).

Brute Force Approach NEW

Try every possible combination of buy/sell day pairs — exponential without structure.

Why We Optimize NEW

With unlimited transactions and no cooldown/fee, the optimal strategy is actually simple and greedy: capture **every single upward price movement** (buy the day before any price increase, sell the day of the increase) — summing `max(0, price[i] - price[i-1])do genuinely need DP because their added restrictions (transaction limits, cooldown, fees) break this simple greedy strategy.`

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with unlimited transactions.

Visual Diagram

```
Prices: [7,1,5,3,6,4]
       Profit: 7 (buy 1 sell 5 = 4, buy 3 sell 6 = 3)
       Total: 7

       Sum all positive differences:
       (5-1)+(6-3) = 4+3=7
```

Solution Approach

Concept: Sum all positive differences.

Algorithm

```
Algorithm MAX_PROFIT_II(prices):
           profit = 0
           For i = 1 to n-1:
               If prices[i] > prices[i-1]:
                   profit += prices[i] - prices[i-1]
           Return profit
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int profit = 0;
               for (int i = 1; i < prices.length; i++) {
                   if (prices[i] > prices[i-1]) {
                        profit += prices[i] - prices[i-1];
                   }
               }
               return profit;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

389. Buy and Sell Stocks III (DP-37)

Understanding the Problem NEW

Same stock setup, but now limited to **at most 2** transactions total.

Brute Force Approach NEW

Try every possible pair of non-overlapping buy-sell windows and compute their combined profit — $O(n^4)$ or worse if done naively by checking all combinations of two transaction windows.

Why We Optimize NEW

Now the transaction limit genuinely requires DP: track state `(day, transactionNumber, holding-or-not)` — for each day, and each of the first/second transaction 'slots,' track the best profit achievable whether you currently hold a stock or not, updating each day based on the previous day's values for buying, selling, or doing nothing. This is where the stock problems stop being solvable by simple greedy summation (as in the unlimited-transactions version, 388) and genuinely need a DP table tracking the extra 'which transaction number, holding or not' state — $O(n)$ time with a small constant state-space multiplier.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with at most 2 transactions.

Visual Diagram

```
Prices: [3,3,5,0,0,3,1,4]
       Max profit: 6 (buy 3 sell 5 = 2, buy 0 sell 4 = 4)

       DP with 2 transactions.
```

Solution Approach

Concept: DP with transaction count.

Algorithm

```
Algorithm MAX_PROFIT_III(prices):
           n = prices.length
           profit = new int[3][n]

           For t = 1 to 2:
               maxDiff = -prices[0]
               For i = 1 to n-1:
                   profit[t][i] = Math.max(profit[t][i-1], prices[i] + maxDiff)
                   maxDiff = Math.max(maxDiff, profit[t-1][i-1] - prices[i])

           Return profit[2][n-1]
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int n = prices.length;
               if (n < 2) return 0;

                 int[][] dp = new int[3][n];

                 for (int t = 1; t <= 2; t++) {
                     int maxDiff = -prices[0];
```

```
                    for (int i = 1; i < n; i++) {
                        dp[t][i] = Math.max(dp[t][i-1], prices[i] + maxDiff);
                        maxDiff = Math.max(maxDiff, dp[t-1][i-1] - prices[i]);
                    }
                }
                return dp[2][n-1];
            }
        }
```

Time & Space Complexity

Time Complexity: O(2*n) | Space Complexity: O(3*n)

390. Buy and Sell Stock IV (DP-38)

Understanding the Problem NEW

Generalizes problem 389 to **at most K** transactions (K given, not fixed at 2).

Brute Force Approach NEW

Same exponential/combinatorial transaction-window exploration as 389, worse for larger K.

Why We Optimize NEW

Direct generalization of 389's DP state: `(day, transactionNumber from 1 to K, holding-or-not)`, with the exact same buy/sell/hold transition logic per day, just extended to K transaction slots instead of a hardcoded 2. Achieves O(n×K) — the same DP shape as 389, parameterized to any K rather than a fixed constant.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with at most k transactions.

Visual Diagram

```
Prices: [3,2,6,5,0,3], k=2
       Max profit: 7 (buy 2 sell 6 = 4, buy 0 sell 3 = 3)

       Same as III but with k.
```

Solution Approach

Concept: DP with k transactions.

Algorithm

```
Algorithm MAX_PROFIT_IV(k, prices):
           n = prices.length
           If k >= n/2: return maxProfit_II(prices)

           dp = new int[k+1][n]
           For t = 1 to k:
               maxDiff = -prices[0]
               For i = 1 to n-1:
                   dp[t][i] = Math.max(dp[t][i-1], prices[i] + maxDiff)
                   maxDiff = Math.max(maxDiff, dp[t-1][i-1] - prices[i])

           Return dp[k][n-1]
```

Java Implementation

```
class Solution {
           public int maxProfit(int k, int[] prices) {
               int n = prices.length;
               if (n < 2) return 0;
               if (k >= n/2) return maxProfitUnlimited(prices);

                 int[][] dp = new int[k+1][n];
                 for (int t = 1; t <= k; t++) {
                     int maxDiff = -prices[0];
                     for (int i = 1; i < n; i++) {
                         dp[t][i] = Math.max(dp[t][i-1], prices[i] + maxDiff);
                         maxDiff = Math.max(maxDiff, dp[t-1][i-1] - prices[i]);
                     }
```

```
                }
                return dp[k][n-1];
            }

            private int maxProfitUnlimited(int[] prices) {
                int profit = 0;
                for (int i = 1; i < prices.length; i++) {
                    if (prices[i] > prices[i-1]) profit += prices[i] - prices[i-1];
                }
                return profit;
            }
        }
```

Time & Space Complexity

Time Complexity: O(k*n) | Space Complexity: O(k*n)

391. Buy and Sell Stocks With Cooldown (DP-39)

Understanding the Problem NEW

Unlimited transactions again, but now with a **cooldown**: after selling, you must wait one day before buying again.

Brute Force Approach NEW

Same exponential exploration without structure as any unconstrained-transaction-sequence problem.

Why We Optimize NEW

Track DP states for each day representing 'holding a stock,' 'not holding, and not in cooldown,' and 'not holding, just sold (in cooldown)' — the cooldown constraint means the 'can buy today' transition must specifically come from the 'not holding, cooldown already over' state, not directly from 'just sold.' This adds one extra state category compared to the simpler unlimited-transactions case, directly modeling the cooldown rule as an explicit state rather than trying to bolt it onto the greedy strategy from problem 388, which can't accommodate this extra day-of-restriction rule.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with cooldown after selling.

Visual Diagram

```
Prices: [1,2,3,0,2]
       Max profit: 3 (buy 1 sell 3 = 2, cooldown, buy 0 sell 2 = 2)
       Total: 3? Actually buy 1 sell 2 = 1, buy 0 sell 2 = 2 total=3

       States:
       hold[i] = max profit holding stock
       sold[i] = max profit sold stock
       rest[i] = max profit in cooldown
```

Solution Approach

Concept: State machine DP.

Algorithm

```
Algorithm MAX_PROFIT_COOLDOWN(prices):
           hold = -prices[0], sold = 0, rest = 0

           For i = 1 to n-1:
               newHold = max(hold, rest - prices[i])
               newSold = hold + prices[i]
               newRest = max(rest, sold)
               hold, sold, rest = newHold, newSold, newRest

           Return max(sold, rest)
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int n = prices.length;
               if (n < 2) return 0;

                 int hold = -prices[0];
```

```
                int sold = 0;
                int rest = 0;

                for (int i = 1; i < n; i++) {
                    int newHold = Math.max(hold, rest - prices[i]);
                    int newSold = hold + prices[i];
                    int newRest = Math.max(rest, sold);
                    hold = newHold;
                    sold = newSold;
                    rest = newRest;
                }
                return Math.max(sold, rest);
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

392. Buy and Sell Stocks With Transaction Fee (DP-40)

Understanding the Problem NEW

Unlimited transactions, but now every *sale* incurs a fixed transaction fee, which must be subtracted from that transaction's profit.

Brute Force Approach NEW

Same exponential exploration without structure.

Why We Optimize NEW

Very similar DP state to the basic unlimited-transactions case (388), tracking 'holding' vs 'not holding' per day, but the profit calculation upon selling now subtracts the fee: `sell profit = price[i] - buyPrice - fee`. This small tweak to the transition (subtract the fee at the point of selling) is enough to correctly handle the fee constraint — worth noting this specific variant sits between the fully-greedy 388 (no state needed at all) and the more state-heavy cooldown version (391), since a flat fee doesn't require an extra 'cooldown' state category, just an adjusted profit calculation.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum profit with transaction fee.

Visual Diagram

```
Prices: [1,3,2,8,4,9], fee=2
       Max profit: 8 (buy 1 sell 8 = 7-2=5, buy 4 sell 9 = 5-2=3)
       Total: 8

       States:
       cash = max profit without stock
       hold = max profit with stock
```

Solution Approach

Concept: State machine DP with fee.

Algorithm

```
Algorithm MAX_PROFIT_FEE(prices, fee):
           cash = 0, hold = -prices[0]

           For i = 1 to n-1:
               cash = Math.max(cash, hold + prices[i] - fee)
               hold = Math.max(hold, cash - prices[i])

           Return cash
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices, int fee) {
               int cash = 0;
               int hold = -prices[0];

                 for (int i = 1; i < prices.length; i++) {
                     cash = Math.max(cash, hold + prices[i] - fee);
```

```
                    hold = Math.max(hold, cash - prices[i]);
                }
                return cash;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

393. Longest Increasing Subsequence (DP-41)

Understanding the Problem NEW

Find the length of the Longest Increasing Subsequence (LIS) in an array — the longest subsequence (not necessarily contiguous) where every element is strictly greater than the previous one.

Brute Force Approach NEW

Try every one of the 2^n subsequences and check which are increasing, tracking the longest — exponential.

Why We Optimize NEW

DP state `lis(i)` — the length of the longest increasing subsequence *ending exactly at index i*: `lis(i) = 1 + max(lis(j))` for every `j < i` where `arr[j] < arr[i]` (or just 1, if no such j exists) — memoized/tabulated per index, giving $O(n^2)$ (checking all previous j for each i). This can be further optimized to O(n log n) using a clever trick: maintain a separate array tracking 'the smallest possible tail value for an increasing subsequence of each length,' and use binary search (from Chapter 3!) to find where each new element fits in — a nice full-circle callback to binary search being useful even in a DP-heavy chapter, whenever a monotonic structure can be exploited.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Length of longest increasing subsequence.

Visual Diagram

```
Array: [10,9,2,5,3,7,101,18]
       LIS = 4 ([2,3,7,101] or [2,5,7,101])

       dp[i] = 1 + max(dp[j]) for j<i and nums[j]<nums[i]
       dp: [1,1,1,2,2,3,4,4]
       Result: 4

       Binary Search Method:
       tails = [10]
       9 → [9]
       2 → [2]
       5 → [2,5]
       3 → [2,3]
       7 → [2,3,7]
       101 → [2,3,7,101]
       18 → [2,3,7,18]
       Length = 4
```

Solution Approach

Concept: DP or binary search.

Algorithm (DP)

Algorithm LIS(nums):

n = nums.length

dp = new int[n]

```
Arrays.fill(dp, 1)

maxLen = 1

For i = 1 to n-1:

For j = 0 to i-1:

If nums[j] < nums[i]:

dp[i] = max(dp[i], dp[j] + 1)

maxLen = max(maxLen, dp[i])

Return maxLen
```

Java Implementation (DP)

```
class Solution {

public int lengthOfLIS(int[] nums) {

int n = nums.length;

int[] dp = new int[n];
```

```
Arrays.fill(dp, 1);

int maxLen = 1;

for (int i = 1; i < n; i++) {

for (int j = 0; j < i; j++) {

if (nums[j] < nums[i]) {

dp[i] = Math.max(dp[i], dp[j] + 1);

}

}

maxLen = Math.max(maxLen, dp[i]);

}

return maxLen;

}

}
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

394. Printing Longest Increasing Subsequence (DP-42)

Understanding the Problem NEW

Same as LIS, but print the actual subsequence, not just its length.

Brute Force Approach NEW

Same exponential subsequence enumeration as plain LIS without a reconstruction plan.

Why We Optimize NEW

While computing the LIS DP table (393), additionally track, for each index, **which previous index** it extended from (whichever j gave the best `lis(j)` value used to compute `lis(i)`) — then after finding the index with the overall maximum LIS length, walk backward through these 'came from' pointers to reconstruct the actual subsequence. Same backward-reconstruction-from-a-filled-table technique as Print LCS (378), applied here to LIS's DP table instead.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print the LIS.

Visual Diagram

```
Array: [10,9,2,5,3,7,101,18]
       LIS: [2,3,7,101]

       Backtrack from dp array.
```

Solution Approach

Concept: DP with parent tracking.

Algorithm

```
Algorithm PRINT_LIS(nums):
           n = nums.length
           dp = new int[n], parent = new int[n]
           Arrays.fill(dp, 1)
           Arrays.fill(parent, -1)
           maxIndex = 0

            For i = 1 to n-1:
                For j = 0 to i-1:
                    If nums[j] < nums[i] && dp[j]+1 > dp[i]:
                        dp[i] = dp[j]+1
                        parent[i] = j
                If dp[i] > dp[maxIndex]: maxIndex = i

            // Backtrack
            lis = []
            i = maxIndex
            While i != -1:
                lis.add(nums[i])
                i = parent[i]
            Return reverse(lis)
```

Java Implementation

```
class Solution {
           public List<Integer> printLIS(int[] nums) {
               int n = nums.length;
               int[] dp = new int[n];
               int[] parent = new int[n];
               Arrays.fill(dp, 1);
               Arrays.fill(parent, -1);
               int maxIndex = 0;

                 for (int i = 1; i < n; i++) {
                     for (int j = 0; j < i; j++) {
                         if (nums[j] < nums[i] && dp[j] + 1 > dp[i]) {
                             dp[i] = dp[j] + 1;
                             parent[i] = j;
                         }
                     }
                     if (dp[i] > dp[maxIndex]) maxIndex = i;
                 }

                 List<Integer> lis = new ArrayList<>();

               int i = maxIndex;
               while (i != -1) {
                   lis.add(nums[i]);
                   i = parent[i];
               }
               Collections.reverse(lis);
               return lis;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

Problem

LIS in O(n log n).

Visual Diagram

```
Array: [10,9,2,5,3,7,101,18]
       tails = []
       10 → [10]
       9 → [9]
       2 → [2]
       5 → [2,5]
       3 → [2,3]
       7 → [2,3,7]
       101 → [2,3,7,101]
       18 → [2,3,7,18]
       Length = 4
```

Solution Approach

Concept: Binary search on tails.

Algorithm

```
Algorithm LIS_BINARY_SEARCH(nums):
           tails = []

            For num in nums:
                pos = lowerBound(tails, num)
                If pos == tails.length:
                    tails.add(num)
                Else:
                    tails[pos] = num

            Return tails.length
```

Java Implementation

```
class Solution {
           public int lengthOfLIS(int[] nums) {
               List<Integer> tails = new ArrayList<>();

                 for (int num : nums) {
                     int pos = Collections.binarySearch(tails, num);
                     if (pos < 0) pos = -(pos + 1);
```

```
                    if (pos == tails.size()) {
                        tails.add(num);
                    } else {
                        tails.set(pos, num);
                    }
                }
                return tails.size();
            }
        }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

396. Largest Divisible Subset (DP-44)

Understanding the Problem NEW

Find the largest subset of an array where every pair of elements satisfies the divisibility property (each element divides, or is divided by, every other element when the subset is sorted).

Brute Force Approach NEW

Try every one of the 2^n subsets and check the pairwise divisibility condition for each — exponential.

Why We Optimize NEW

After **sorting** the array, this becomes structurally identical to LIS (393): a valid divisible subset, once sorted, is exactly a subsequence where each next element is *divisible by* the previous one (analogous to LIS's 'greater than' condition) — reusing the exact same `dp(i) = 1 + max(dp(j))` recurrence, just replacing the 'greater than' comparison with 'divisible by.' Sorting first is what makes this reduction valid, echoing a similar 'sort first, then apply a familiar recurrence' pattern seen in several Greedy-chapter problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Largest subset where every pair is divisible.

Visual Diagram

```
Array: [1,2,3,4,8]
       LDS: [1,2,4,8] (or [1,3])

       Sort array.
       dp[i] = largest subset ending at i
       If nums[i] % nums[j] == 0: dp[i] = max(dp[i], dp[j]+1)
```

Solution Approach

Concept: DP on sorted array.

Algorithm

```
Algorithm LARGEST_DIVISIBLE_SUBSET(nums):
           Sort(nums)
           n = nums.length
           dp = new int[n]
           parent = new int[n]
           Arrays.fill(dp, 1)
           Arrays.fill(parent, -1)
           maxIndex = 0

           For i = 1 to n-1:
               For j = 0 to i-1:
                   If nums[i] % nums[j] == 0 && dp[j]+1 > dp[i]:
                       dp[i] = dp[j]+1
                       parent[i] = j
               If dp[i] > dp[maxIndex]: maxIndex = i

           // Backtrack
           subset = []
           i = maxIndex
           While i != -1:
               subset.add(nums[i])
               i = parent[i]
           Return reverse(subset)
```

Java Implementation

```
class Solution {
           public List<Integer> largestDivisibleSubset(int[] nums) {
               Arrays.sort(nums);
               int n = nums.length;
               int[] dp = new int[n];
               int[] parent = new int[n];
               Arrays.fill(dp, 1);
               Arrays.fill(parent, -1);
               int maxIndex = 0;

                 for (int i = 1; i < n; i++) {
                     for (int j = 0; j < i; j++) {
                         if (nums[i] % nums[j] == 0 && dp[j] + 1 > dp[i]) {
                             dp[i] = dp[j] + 1;
                             parent[i] = j;
                         }

                    }
                    if (dp[i] > dp[maxIndex]) maxIndex = i;
               }

               List<Integer> result = new ArrayList<>();
               int i = maxIndex;
               while (i != -1) {
                   result.add(nums[i]);
                   i = parent[i];
               }
               Collections.reverse(result);
               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

397. Longest String Chain (DP-45)

Understanding the Problem NEW

Given a list of words, find the longest chain where each word in the chain is formed by adding exactly one character to the previous word (in any position), and each intermediate word must also exist in the given list.

Brute Force Approach NEW

Try every possible ordering/subset of words and check the chain-forming condition — exponential.

Why We Optimize NEW

After **sorting words by length**, this becomes an LIS-shaped problem again: `dp(word) = 1 + dp(predecessor)` where 'predecessor' is any word in the list that's exactly one character shorter and forms the current word when one character is added — checked by trying every possible single-character removal from the current word and looking it up (e.g. via a hashmap of word → best chain length). Same 'sort first, then apply an LIS-shaped recurrence over a customized 'compatibility' condition instead of simple numeric comparison' pattern as problem 396, just with a string-based 'predecessor' check instead of divisibility.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Longest chain where each string differs by one char.

Visual Diagram

```
Words: ["a","b","ba","bca","bda","bdca"]
       Longest chain: ["a","ba","bda","bdca"] length=4

       Sort by length
       dp[i] = 1 + max(dp[j]) where s[j] is predecessor
```

Solution Approach

Concept: DP on sorted strings.

Algorithm

```
Algorithm LONGEST_STRING_CHAIN(words):
           Sort by length
           dp = new int[n]
           Arrays.fill(dp, 1)
           maxLen = 1

           For i = 1 to n-1:
               For j = 0 to i-1:
                   If isPredecessor(words[j], words[i]):
                       dp[i] = max(dp[i], dp[j]+1)
               maxLen = max(maxLen, dp[i])

           Return maxLen

       Algorithm isPredecessor(s1, s2):
           If s2.length != s1.length+1: Return false
           i = j = 0
           diff = 0
           While i < s1.length && j < s2.length:
               If s1[i] != s2[j]:
                    diff++
                    j++
```

```
                Else:
                     i++; j++
           Return diff <= 1
```

Java Implementation

```
class Solution {
           public int longestStrChain(String[] words) {
               Arrays.sort(words, (a, b) -> a.length() - b.length());
               int n = words.length;
               int[] dp = new int[n];
               Arrays.fill(dp, 1);
               int maxLen = 1;

                 for (int i = 1; i < n; i++) {
                     for (int j = 0; j < i; j++) {
                         if (isPredecessor(words[j], words[i])) {
                             dp[i] = Math.max(dp[i], dp[j] + 1);
                         }
                     }
                     maxLen = Math.max(maxLen, dp[i]);

               }
               return maxLen;
           }

           private boolean isPredecessor(String s1, String s2) {
               if (s2.length() != s1.length() + 1) return false;
               int i = 0, j = 0;
               while (i < s1.length() && j < s2.length()) {
                   if (s1.charAt(i) == s2.charAt(j)) {
                       i++;
                       j++;
                   } else {
                       j++;
                   }
               }
               return i == s1.length();
           }
       }
```

Time & Space Complexity

Time Complexity: O(n² * L) | Space Complexity: O(n)

398. Longest Bitonic Subsequence (DP-46)

Understanding the Problem NEW

Find the longest 'bitonic' subsequence — one that strictly increases and then strictly decreases (like a mountain shape), possibly with only an increasing or only a decreasing part.

Brute Force Approach NEW

Try every subsequence and check the increase-then-decrease shape — exponential.

Why We Optimize NEW

Compute two LIS-style DP tables: one for the **longest increasing subsequence ending at each index** (exactly problem 393's DP, left to right), and one for the **longest decreasing subsequence starting at each index** (the same DP, but scanning right to left, or equivalently LIS on the reversed array) — then for each index, the best bitonic sequence 'peaking' there is `(increasing length ending here) + (decreasing length starting here) - 1` (subtracting 1 to avoid double-counting the peak element itself), and the answer is the maximum over all indices. This combines two separately-computed LIS tables — computed left-to-right and right-to-left — directly reusing the LIS recurrence twice rather than designing a new algorithm for the mountain shape.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Longest increasing then decreasing subsequence.

Visual Diagram

```
Array: [1, 11, 2, 10, 4, 5, 2, 1]
       LBS: [1, 2, 10, 4, 2, 1] length=6

       Compute:
       lis[i] = LIS ending at i
       lds[i] = LDS starting at i
       bitonic[i] = lis[i] + lds[i] - 1
```

Solution Approach

Concept: LIS + LDS.

Algorithm

```
Algorithm BITONIC_SUBSEQUENCE(nums):
           n = nums.length
           lis = LIS_ending_at_each(nums)
           lds = LDS_starting_at_each(nums)

           maxLen = 0
           For i = 0 to n-1:
               maxLen = max(maxLen, lis[i] + lds[i] - 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int longestBitonicSubsequence(int[] nums) {
               int n = nums.length;
```

```
                int[] lis = new int[n];
                int[] lds = new int[n];
                Arrays.fill(lis, 1);
                Arrays.fill(lds, 1);

                  // LIS ending at i
                  for (int i = 1; i < n; i++) {
                      for (int j = 0; j < i; j++) {
                          if (nums[j] < nums[i]) {
                              lis[i] = Math.max(lis[i], lis[j] + 1);
                          }
                      }
                  }

                  // LDS starting at i
                  for (int i = n-2; i >= 0; i--) {
                      for (int j = n-1; j > i; j--) {
                          if (nums[j] < nums[i]) {
                              lds[i] = Math.max(lds[i], lds[j] + 1);
                          }
                      }
                  }

                  int maxLen = 0;
                  for (int i = 0; i < n; i++) {

                    maxLen = Math.max(maxLen, lis[i] + lds[i] - 1);
                }
                return maxLen;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

399. Number of Longest Increasing Subsequences (DP-47)

Understanding the Problem NEW

Count how many distinct Longest Increasing Subsequences exist (not just find the length of one, but count all subsequences that achieve that maximum length).

Brute Force Approach NEW

Enumerate every subsequence, filter for increasing ones matching the maximum length, and count — exponential.

Why We Optimize NEW

Extend the LIS DP (393) with a parallel **count array**: alongside `lis(i)` (the length), track `count(i)` (how many distinct subsequences of that maximum length end at index i) — when extending from a previous index j, if `lis(j)+1` creates a *new best* length at i, reset `count(i)` to `count(j)`; if it *ties* the current best length at i, **add** `count(j)` to `count(i)` (multiple ways to reach the same best length). Sum `count(i)` over every index that achieves the overall maximum LIS length. This is the same 'add a parallel counting array alongside the main DP table' technique seen in graph shortest-path counting (Number of Ways to Arrive at Destination, problem 334) — applied here to LIS instead of shortest paths.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count of LIS.

Visual Diagram

```
Array: [1,3,5,4,7]
       LIS length = 3, count = 2 ([1,3,5,7] and [1,3,4,7])

       dp[i] = length of LIS ending at i
       count[i] = number of LIS ending at i
```

Solution Approach

Concept: DP with length and count.

Algorithm

```
Algorithm COUNT_LIS(nums):
           n = nums.length
           dp = new int[n]
           count = new int[n]
           Arrays.fill(dp, 1)
           Arrays.fill(count, 1)
           maxLen = 1

           For i = 1 to n-1:
               For j = 0 to i-1:
                   If nums[j] < nums[i]:
                       If dp[j] + 1 > dp[i]:
                           dp[i] = dp[j] + 1
                           count[i] = count[j]
                       Else If dp[j] + 1 == dp[i]:
                           count[i] += count[j]
               maxLen = max(maxLen, dp[i])

           total = 0
           For i = 0 to n-1:
               If dp[i] == maxLen:
```

```
                total += count[i]
        Return total
```

Java Implementation

```
class Solution {
        public int findNumberOfLIS(int[] nums) {
            int n = nums.length;
            int[] dp = new int[n];
            int[] count = new int[n];
            Arrays.fill(dp, 1);
            Arrays.fill(count, 1);
            int maxLen = 1;

              for (int i = 1; i < n; i++) {
                  for (int j = 0; j < i; j++) {
                      if (nums[j] < nums[i]) {
                          if (dp[j] + 1 > dp[i]) {
                              dp[i] = dp[j] + 1;
                              count[i] = count[j];
                          } else if (dp[j] + 1 == dp[i]) {
                              count[i] += count[j];

                           }
                       }
                  }
                  maxLen = Math.max(maxLen, dp[i]);
              }

              int total = 0;
              for (int i = 0; i < n; i++) {
                  if (dp[i] == maxLen) {
                      total += count[i];
                  }
              }
              return total;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

400. Matrix Chain Multiplication (DP-48)

Understanding the Problem NEW

Given a chain of matrices to multiply together, find the minimum number of scalar multiplications needed, by choosing the optimal order/parenthesization to multiply them in (matrix multiplication is associative, so different groupings give the same result but very different costs).

Brute Force Approach NEW

Try every possible way to parenthesize the matrix chain — this follows the Catalan number sequence, growing exponentially with the number of matrices.

Why We Optimize NEW

DP state `minCost(i, j)` — the minimum cost to multiply matrices i through j: try every possible **split point k** between i and j, computing `minCost(i,k) + minCost(k+1,j) + (cost of multiplying the two resulting combined matrices)`, taking the minimum over every possible k. This is a genuinely new DP **shape** for this chapter — an 'interval DP,' where the state is a *range* (i,j) rather than a single index or index-pair-between-two-sequences — memoized/tabulated over all $O(n^2)$ possible (i,j) ranges, each trying $O(n)$ split points, giving $O(n^3)$ instead of the exponential Catalan-number-scaling brute force. This interval-DP shape (state = a range, transition = try every split point) becomes the template for several remaining 'hard'

> problems in this chapter (Cutting a Stick, Burst Balloons, Palindrome Partitioning-II).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum scalar multiplications to multiply matrices.

Visual Diagram

```
Dimensions: [10, 30, 5, 60]
       Matrices: A1(10x30), A2(30x5), A3(5x60)
       Min: 4500
       Order: (A1*A2)*A3 = 10*30*5 + 10*5*60 = 1500+3000=4500

       dp[i][j] = min cost to multiply matrices i to j
       dp[i][j] = min(dp[i][k] + dp[k+1][j] + dims[i]*dims[k+1]*dims[j+1])
```

Solution Approach

Concept: Partition DP.

Algorithm

```
Algorithm MCM(dims):
           n = dims.length - 1
           dp = int[n][n]

           For len = 2 to n:
               For i = 0 to n-len:
                   j = i + len - 1
                   dp[i][j] = INF
                   For k = i to j-1:
                       cost = dp[i][k] + dp[k+1][j] + dims[i]*dims[k+1]*dims[j+1]
                       dp[i][j] = min(dp[i][j], cost)
```

```
            Return dp[0][n-1]
```

Java Implementation

```
class Solution {
            public int matrixChainOrder(int[] dims) {
                int n = dims.length - 1;
                int[][] dp = new int[n][n];

                  for (int len = 2; len <= n; len++) {
                      for (int i = 0; i <= n - len; i++) {
                          int j = i + len - 1;
                          dp[i][j] = Integer.MAX_VALUE;
                          for (int k = i; k < j; k++) {
                              int cost = dp[i][k] + dp[k+1][j] + dims[i] * dims[k+1]
* dims[j+1];
                              dp[i][j] = Math.min(dp[i][j], cost);
                          }
                      }
                  }
                  return dp[0][n-1];
            }
       }
```

Time & Space Complexity

Time Complexity: O(n³) | Space Complexity: O(n²)

401. Matrix Chain Multiplication Bottom-Up (DP-49)

Understanding the Problem NEW

The exact same Matrix Chain Multiplication problem (400), solved bottom-up (tabulation) instead of top-down (memoization).

Brute Force Approach NEW

Same exponential Catalan-scaling brute force as 400 without any DP.

Why We Optimize NEW

Fill the `minCost(i,j)` table by **increasing interval length** — first all length-1 ranges (trivially cost 0, a single matrix), then length-2, then length-3, and so on, since computing any `minCost(i,j)` requires smaller sub-ranges to already be filled in. This bottom-up ordering (process by increasing range size, rather than recursing top-down) is a general pattern for tabulating *any* interval DP, worth remembering for problems 402-406 which share this same interval-DP shape.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

MCM bottom-up approach.

Visual Diagram

```
Same as above with iterative DP.
```

Java Implementation

```
class Solution {
           public int matrixChainOrder(int[] dims) {
               int n = dims.length;
               int[][] dp = new int[n][n];

               for (int len = 2; len < n; len++) {
                   for (int i = 1; i < n - len + 1; i++) {
                       int j = i + len - 1;
                       dp[i][j] = Integer.MAX_VALUE;
                       for (int k = i; k < j; k++) {
                           int cost = dp[i][k] + dp[k+1][j] + dims[i-1] * dims[k] *
dims[j];
                           dp[i][j] = Math.min(dp[i][j], cost);
                       }
                   }
               }
               return dp[1][n-1];
           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$

402. Minimum Cost to Cut the Stick (DP-50)

Understanding the Problem NEW

Given a stick of a certain length and a set of positions where cuts must be made, find the minimum total cost to make all the cuts — where each individual cut costs the length of the piece being cut at that moment (so cutting order matters a lot).

Brute Force Approach NEW

Try every possible order of making the cuts and compute the total cost for each — n! orderings, factorial and infeasible for more than a few cuts.

Why We Optimize NEW

Same interval-DP shape as Matrix Chain Multiplication: `minCost(left, right)` — the minimum cost to make all required cuts strictly between positions left and right — tries every possible cut position `c` within that range as the *first* cut made in that sub-range, paying `(right - left)` for that cut (the current piece's length) plus the cost of recursively handling the two resulting sub-pieces. This directly reuses the 'range as state, try every split point' interval-DP template from Matrix Chain Multiplication, achieving $O(n^3)$ instead of the factorial brute-force ordering search.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum cost to cut a stick at given positions.

Visual Diagram

```
n=7, cuts=[1,3,4,5]
       Min cost = 16

       Add 0 and n to cuts, sort.
       dp[i][j] = min cost to cut between cuts[i] and cuts[j]
       dp[i][j] = min(dp[i][k] + dp[k][j] + (cuts[j]-cuts[i]))
```

Solution Approach

Concept: Partition DP.

Algorithm

```
Algorithm MIN_COST_CUT(n, cuts):
           cuts = [0] + sort(cuts) + [n]
           m = cuts.length
           dp = int[m][m]

           For len = 2 to m-1:
               For i = 0 to m-len-1:
                   j = i + len
                   dp[i][j] = INF
                   For k = i+1 to j-1:
                       dp[i][j] = min(dp[i][j], dp[i][k] + dp[k][j] + cuts[j] -
cuts[i])

           Return dp[0][m-1]
```

Java Implementation

```
class Solution {
           public int minCost(int n, int[] cuts) {
               int m = cuts.length + 2;
               int[] newCuts = new int[m];
               newCuts[0] = 0;
               newCuts[m-1] = n;
               System.arraycopy(cuts, 0, newCuts, 1, cuts.length);
               Arrays.sort(newCuts);

                 int[][] dp = new int[m][m];
                 for (int len = 2; len < m; len++) {
                     for (int i = 0; i < m - len; i++) {
                         int j = i + len;
                         dp[i][j] = Integer.MAX_VALUE;
                         for (int k = i + 1; k < j; k++) {
                             dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k][j] +
newCuts[j] - newCuts[i]
                         }
                     }
                 }
                 return dp[0][m-1];
           }
       }
```

Time & Space Complexity

Time Complexity: O(m³) | Space Complexity: O(m²)

403. Burst Balloons (DP-51)

Understanding the Problem NEW

Given balloons each with a value, bursting a balloon earns coins equal to the product of its value and its two current neighbours' values (after bursting, the balloon's former neighbours become adjacent to each other); find the maximum total coins from bursting all balloons in some order.

Brute Force Approach NEW

Try every possible order of bursting all balloons — n! orderings, factorial and infeasible.

Why We Optimize NEW

The trick that makes this an interval DP: instead of thinking 'which balloon do I burst first,' think '**which balloon do I burst LAST** within a given range' — because the last balloon burst in a range has, by definition, the two range-boundary balloons as its final neighbours (since everything else in the range is already gone). DP state `maxCoins(left, right)` tries every possible balloon `k` as the 'last burst' within that range, computing `maxCoins(left,k) + maxCoins(k,right) + value[left]×value[k]×value[right]`. This 'think about what happens last, not first' reframing (rather than the more intuitive 'first choice' framing that doesn't decompose cleanly) is the key insight, still fitting the same interval-DP template as Matrix Chain Multiplication and Cutting a Stick, achieving $O(n^3)$ instead of factorial ordering exploration.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Max coins by bursting balloons.

Visual Diagram

```
Balloons: [3,1,5,8]
       Max coins: 167
       Order: 1, 5, 3, 8

       Add 1 at both ends: [1,3,1,5,8,1]
       dp[i][j] = max coins from bursting i+1..j-1
       dp[i][j] = max(dp[i][k] + dp[k][j] + nums[i]*nums[k]*nums[j])
```

Solution Approach

Concept: Partition DP.

Algorithm

```
Algorithm MAX_COINS(nums):
           n = nums.length
           newNums = [1] + nums + [1]
           dp = int[n+2][n+2]

           For len = 1 to n:
               For i = 1 to n-len+1:
                   j = i + len - 1
                   For k = i to j:
                       dp[i][j] = max(dp[i][j], dp[i][k-1] + dp[k+1][j] +
newNums[i-1]*newNums[k]*newN

           Return dp[1][n]
```

Java Implementation

```
class Solution {
           public int maxCoins(int[] nums) {
               int n = nums.length;
               int[] newNums = new int[n + 2];
               newNums[0] = 1;
               newNums[n+1] = 1;
               System.arraycopy(nums, 0, newNums, 1, n);

                 int[][] dp = new int[n+2][n+2];
                 for (int len = 1; len <= n; len++) {
                     for (int i = 1; i <= n - len + 1; i++) {
                         int j = i + len - 1;
                         for (int k = i; k <= j; k++) {
                             dp[i][j] = Math.max(dp[i][j],
                                  dp[i][k-1] + dp[k+1][j] + newNums[i-1] *
newNums[k] * newNums[j+1]);
                         }
                     }
                 }
                 return dp[1][n];
           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$

404. Evaluate Boolean Expression to True (DP-52)

Understanding the Problem NEW

Given a boolean expression with operands (True/False) and operators (AND, OR, XOR), count the number of ways to parenthesize the expression such that it evaluates to True.

Brute Force Approach NEW

Try every possible parenthesization — following the same Catalan-number exponential growth as Matrix Chain Multiplication's naive approach.

Why We Optimize NEW

Interval DP again, but tracking **two** counts per range instead of one: `waysTrue(left,right)` and `waysFalse(left,right)` — for every split point (every operator position) within the range, compute how many ways the left and right sub-expressions can each evaluate to True or False, then combine them according to the specific operator's truth table (e.g. for OR, the only way to get False is both sides False; every other combination gives True) to determine how many combined ways yield True vs False overall. This extends the interval-DP template with a 'track two parallel counts instead of one value' twist — similar in spirit to how LIS-counting (399) added a parallel count array alongside its main DP value.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Ways to evaluate expression to True.

Visual Diagram

```
s = "T|F", result=true
       Ways: 1

       dp[i][j][0/1] = ways to get False/True
       If operator == '|':
           True = leftTrue*rightTrue + leftTrue*rightFalse + leftFalse*rightTrue
           False = leftFalse*rightFalse
```

Solution Approach

Concept: Partition DP.

Algorithm

```
Algorithm BOOLEAN_EVALUATION(s, result):
           n = s.length
           dp = new int[n][n][2]

           For i = 0 to n-1 step 2:
               dp[i][i][1] = s[i] == 'T' ? 1 : 0
               dp[i][i][0] = s[i] == 'F' ? 1 : 0

           For len = 3 to n step 2:
               For i = 0 to n-len step 2:
                   j = i + len - 1
                   For k = i+1 to j-1 step 2:
                       For op in [T,F]:
                           // Count ways for each operator
           Return dp[0][n-1][1]
```

Java Implementation

```
class Solution {
           public int countWays(String s) {
               int n = s.length();
               int[][][] dp = new int[n][n][2];

                 for (int i = 0; i < n; i += 2) {
                     dp[i][i][0] = s.charAt(i) == 'F' ? 1 : 0;
                     dp[i][i][1] = s.charAt(i) == 'T' ? 1 : 0;
                 }

                 for (int len = 3; len <= n; len += 2) {
                     for (int i = 0; i <= n - len; i += 2) {
                         int j = i + len - 1;
                         for (int k = i + 1; k < j; k += 2) {
                             char op = s.charAt(k);
                             int lt = dp[i][k-1][1], lf = dp[i][k-1][0];
                             int rt = dp[k+1][j][1], rf = dp[k+1][j][0];

                             if (op == '|') {
                                 dp[i][j][1] += lt*rt + lt*rf + lf*rt;
                                 dp[i][j][0] += lf*rf;
                             } else if (op == '&') {
                                 dp[i][j][1] += lt*rt;

                               dp[i][j][0] += lf*rf + lt*rf + lf*rt;
                           } else { // '^'
                               dp[i][j][1] += lt*rf + lf*rt;
                               dp[i][j][0] += lt*rt + lf*rf;
                           }
                       }
                   }
               }
               return dp[0][n-1][1];
           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$

405. Palindrome Partitioning - II (DP-53)

Understanding the Problem NEW

Find the minimum number of cuts needed to partition a string into pieces that are all palindromes.

Brute Force Approach NEW

Try every possible way to place cuts and check if every resulting piece is a palindrome, tracking the minimum cuts — exponential (similar to the Recursion chapter's Palindrome Partitioning problem, 143, but this version only needs the minimum cut *count*, not every possible partition).

Why We Optimize NEW

DP state `minCuts(i)` — the minimum cuts needed for the first i characters: for every possible last piece (from some start position j to i), if that substring is a palindrome (precomputed once via a separate O(n²) palindrome-checking table, so each check is O(1) rather than re-verified from scratch), then `minCuts(i) = min(minCuts(j-1) + 1)` over all valid j. This connects back to the Recursion chapter's backtracking approach (143, which explored every valid partition) by instead just tracking the minimum cut count via DP — a good example of a problem sitting on the boundary between 'backtracking generates every valid answer' (Recursion chapter) and 'DP finds just the optimal count' (this chapter), achieving O(n²) instead of the recursion chapter's more exploratory approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum cuts for palindrome partitions.

Visual Diagram

```
s = "aab"
       Min cuts: 1 ("aa","b")

       dp[i] = min cuts for s[0..i]
       If palindrome: dp[i] = 0
       Else: dp[i] = min(dp[j] + 1) for j<i and s[j+1..i] palindrome
```

Solution Approach

Concept: DP with palindrome check.

Algorithm

```
Algorithm MIN_CUTS(s):
           n = s.length()
           isPal = boolean[n][n]
           dp = new int[n]

           For len = 1 to n:
               For i = 0 to n-len:
                   j = i + len - 1
                   If s[i] == s[j] && (len <= 2 || isPal[i+1][j-1]):
                       isPal[i][j] = true

           For i = 0 to n-1:
               If isPal[0][i]:
                   dp[i] = 0
               Else:
                   dp[i] = INF
                   For j = 0 to i-1:
                       If isPal[j+1][i]:
```

```
                            dp[i] = min(dp[i], dp[j] + 1)

            Return dp[n-1]
```

Java Implementation

```
class Solution {
            public int minCut(String s) {
                int n = s.length();
                boolean[][] isPal = new boolean[n][n];

                  for (int len = 1; len <= n; len++) {
                      for (int i = 0; i <= n - len; i++) {
                          int j = i + len - 1;
                          if (s.charAt(i) == s.charAt(j) && (len <= 2 || isPal[i+1]
[j-1])) {
                              isPal[i][j] = true;
                          }
                      }
                  }

                  int[] dp = new int[n];
                  for (int i = 0; i < n; i++) {
                      if (isPal[0][i]) {
                          dp[i] = 0;

                    } else {
                        dp[i] = Integer.MAX_VALUE;
                        for (int j = 0; j < i; j++) {
                             if (isPal[j+1][i]) {
                                 dp[i] = Math.min(dp[i], dp[j] + 1);
                             }
                        }
                    }
                }
                return dp[n-1];
            }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n²)

406. Partition Array for Maximum Sum (DP-54)

Understanding the Problem NEW

Partition an array into contiguous subarrays of length at most K, where each subarray's elements all get replaced by that subarray's maximum value; maximize the total sum after this transformation.

Brute Force Approach NEW

Try every possible way to partition the array into valid-length groups — exponential without structure, since there are many ways to place partition boundaries.

Why We Optimize NEW

DP state `maxSum(i)` — the best achievable sum for the first i elements: try every possible **last group length** from 1 to K ending at position i, computing `maxSum(i) = max over length L of (maxSum(i-L) + L × max(last L elements))`. This is a 1D DP (unlike the interval DPs of 400-404) but still requires trying multiple 'how far back does the last group extend' options at each position — similar in flavor to the K-distance Frog Jump (356), just with a more involved per-position computation (finding the max of the last L elements) rather than a simple cost lookup.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Partition array to maximize sum of max * length.

Visual Diagram

```
Array: [1,15,7,9,2,5,10], k=3
       Max sum: 84

       dp[i] = max sum for first i elements
       dp[i] = max(dp[i-k] + max * k) for k=1..min(k,i)
```

Solution Approach

Concept: DP with sliding window.

Algorithm

```
Algorithm PARTITION_MAX_SUM(arr, k):
           n = arr.length
           dp = new int[n+1]

           For i = 1 to n:
               maxVal = 0
               For j = i-1 down to max(0, i-k):
                   maxVal = max(maxVal, arr[j])
                   dp[i] = max(dp[i], dp[j] + maxVal * (i-j))

           Return dp[n]
```

Java Implementation

```
class Solution {
           public int maxSumAfterPartitioning(int[] arr, int k) {
               int n = arr.length;
               int[] dp = new int[n+1];

                 for (int i = 1; i <= n; i++) {
```

```
                    int maxVal = 0;
                    for (int j = i-1; j >= Math.max(0, i-k); j--) {
                        maxVal = Math.max(maxVal, arr[j]);
                        dp[i] = Math.max(dp[i], dp[j] + maxVal * (i-j));
                    }
                }
                return dp[n];
            }
        }
```

Time & Space Complexity

Time Complexity: O(n*k) | Space Complexity: O(n)

407. Maximum Rectangle Area with all 1's (DP-55)

Understanding the Problem NEW

Given a binary matrix, find the area of the largest rectangle containing only 1s — the same problem as the Stack/Queue chapter's Maximal Rectangle (problem 193), revisited here to connect it to this chapter's DP techniques.

Brute Force Approach NEW

Check every possible rectangle directly — extremely expensive, as flagged back in problem 193.

Why We Optimize NEW

As established in problem 193: treat each row as the 'ground level' of a histogram (building a running 'height' for each column based on consecutive 1s above), and apply the Largest Rectangle in Histogram technique (a monotonic-stack technique from the Stack/Queue chapter, problem 192) to each row's histogram — O(rows × cols) total. This problem is placed here specifically to highlight that not every 'hard' problem in the DP chapter needs a DP solution — sometimes recognizing a connection to a technique from an entirely different chapter (monotonic stacks) is the real insight, and forcing a DP-only lens onto every problem in a DP chapter would be a mistake.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Largest rectangle of 1s in binary matrix.

Visual Diagram

```
Matrix:
       1 0 1 0 0
       1 0 1 1 1
       1 1 1 1 1
       1 0 0 1 0

       Max area: 6 (rows 1-2, cols 2-4)

       Convert to histogram heights:
       Row0: [1,0,1,0,0]
       Row1: [2,0,2,1,1]
       Row2: [3,1,3,2,2]
       Row3: [4,0,0,3,0]
       Max rectangle in histogram = 6
```

Solution Approach

Concept: Histogram DP.

Algorithm

```
Algorithm MAXIMAL_RECTANGLE(matrix):
           m = matrix.length, n = matrix[0].length
           heights = new int[n]
           maxArea = 0

            For i = 0 to m-1:
                For j = 0 to n-1:
                    If matrix[i][j] == '1':
                        heights[j]++
                    Else:
                        heights[j] = 0
                maxArea = max(maxArea, largestRectangleArea(heights))
```

```
            Return maxArea
```

Java Implementation

```
class Solution {
           public int maximalRectangle(char[][] matrix) {
               if (matrix.length == 0) return 0;
               int m = matrix.length, n = matrix[0].length;
               int[] heights = new int[n];
               int maxArea = 0;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         heights[j] = matrix[i][j] == '1' ? heights[j] + 1 : 0;
                     }
                     maxArea = Math.max(maxArea, largestRectangleArea(heights));
                 }
                 return maxArea;

          }

          private int largestRectangleArea(int[] heights) {
              Stack<Integer> stack = new Stack<>();
              int maxArea = 0;
              int n = heights.length;

              for (int i = 0; i <= n; i++) {
                  int h = (i == n) ? 0 : heights[i];
                  while (!stack.isEmpty() && heights[stack.peek()] > h) {
                      int height = heights[stack.pop()];
                      int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                      maxArea = Math.max(maxArea, height * width);
                  }
                  stack.push(i);
              }
              return maxArea;
          }
      }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(n)

408. Count Square Submatrices with All Ones (DP-56)

Understanding the Problem NEW

Given a binary matrix, count the total number of square submatrices consisting entirely of 1s (of any size, from 1×1 up to the largest possible).

Brute Force Approach NEW

Check every possible square (every top-left corner and every possible size) directly — O((rows×cols)²) or worse.

Why We Optimize NEW

DP state `largestSquareEndingAt(r,c)` — the size of the largest all-1s square with its bottom-right corner at (r,c): if the current cell is a 1, `dp(r,c) = 1 + min(dp(r-1,c), dp(r,c-1), dp(r-1,c-1))` (limited by the smallest of the three neighboring squares, since all three must support at least that size for a bigger square to form) — and critically, **the value at each cell directly tells you how many valid squares have their bottom-right corner there** (a cell with dp value 3 means there are 3 valid squares — sizes 1, 2, and 3 — ending there), so summing `dp(r,c)` over the entire grid gives the total count directly. This closing problem elegantly reuses the classic 'largest square ending here' DP recurrence, with the added observation that the DP *value itself* (not just its maximum) directly answers a counting question — a fitting note to end the chapter on, given how often this book has shown the same DP table serving

> multiple related questions (length, count, or reconstruction) with only a small twist in how the final answer is read off.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count all square submatrices of 1s.

Visual Diagram

```
Matrix:
       [1,1,1]
       [1,1,1]
       [1,1,1]

       Squares:
       1x1: 9
       2x2: 4
       3x3: 1
       Total: 14

       dp[i][j] = size of largest square ending at (i,j)
       If matrix[i][j] == 1:
           dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
       Else: dp[i][j] = 0
```

Solution Approach

Concept: DP with min of neighbors.

Algorithm

```
Algorithm COUNT_SQUARES(matrix):
           m = matrix.length, n = matrix[0].length
           dp = int[m][n]
           total = 0
```

```
            For i = 0 to m-1:
                For j = 0 to n-1:
                    If matrix[i][j] == 1:
                        If i == 0 || j == 0:
                            dp[i][j] = 1
                        Else:
                            dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
                        total += dp[i][j]

            Return total
```

Java Implementation

```
class Solution {
            public int countSquares(int[][] matrix) {
                int m = matrix.length, n = matrix[0].length;
                int[][] dp = new int[m][n];
                int total = 0;

                  for (int i = 0; i < m; i++) {
                      for (int j = 0; j < n; j++) {
                          if (matrix[i][j] == 1) {
                              if (i == 0 || j == 0) {
                                  dp[i][j] = 1;
                              } else {
                                  dp[i][j] = 1 + Math.min(dp[i-1][j],
                                              Math.min(dp[i][j-1], dp[i-1][j-1]));
                              }

                               total += dp[i][j];
                           }
                       }
                   }
                   return total;
               }
          }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(m*n)

Dynamic Programming Summary

Problem	Category	Time	Space	Key Technique
Fibonacci/Intro	1D DP	O(n)	O(1)	DP basics
Climbing Stairs	1D DP	O(n)	O(1)	Fibonacci pattern
Frog Jump	1D DP	O(n)	O(n)	Two-step transition
Frog Jump K	1D DP	O(n*k)	O(n)	K-step transition
Max Non-Adjacent	1D DP	O(n)	O(1)	Include/Exclude
House Robber	1D DP	O(n)	O(1)	Include/Exclude
Ninja Training	2D DP	O(n)	O(n*3)	Activity constraint
Unique Paths	Grid	O(m*n)	O(m*n)	Left + Top
Min Path Sum	Grid	O(m*n)	O(m*n)	Min of neighbors
Falling Path	Grid	O(m*n)	O(m*n)	Three directions
Subset Sum	Subsequence	O(n*target)	O(target)	Include/Exclude
Equal Subset	Subsequence	O(n*target)	O(target)	Target sum
Knapsack	Subsequence	O(n*W)	O(W)	Weight capacity
Min Coins	Subsequence	O(amount*coins)	O(amount)	Unbounded
Coin Change 2	Subsequence	O(coins*amount)	O(amount)	Unbounded count
LCS	Strings	O(m*n)	O(m*n)	Match/Skip
Edit Distance	Strings	O(m*n)	O(m*n)	Delete/Insert/ Replace
Wildcard Matching	Strings	O(m*n)	O(m*n)	Pattern matching
Stock I	Stocks	O(n)	O(1)	Min price
Stock II	Stocks	O(n)	O(1)	Positive differences
Stock IV	Stocks	O(k*n)	O(k*n)	DP with transactions
LIS	LIS	$O(n^2)$	O(n)	DP on indices
LIS Binary	LIS	O(n log n)	O(n)	Binary search
String Chain	LIS	$O(n^2*L)$	O(n)	Predecessor check
MCM	Partition	$O(n^3)$	$O(n^2)$	Interval DP

Burst Balloons	Partition	$O(n^3)$	$O(n^2)$	Interval DP
Max Rectangle	Squares	O(m*n)	O(n)	Histogram DP
Count Squares	Squares	O(m*n)	O(m*n)	Min of neighbors

— End of enhanced Chapter 13: Dynamic Programming —